MINIMAX-H3 / TWO-GPU INFERENCE

# Integrated Optimization Technical Guide

An implementation-level explanation of attention, INT8 row operators, TP2 communication, LoRA, memory residency and parallel VAEs.

| TE | DiT | Video VAE |
| --- | --- | --- |
| Group32 dense | Kitchen INT8 | Kitchen + Q split |

CURRENT SOURCE, NOT A BLENDED BENCHMARK CLAIM

FP16-fix residuals. Fused FP32 SwiGLU / ConvRot preparation. Scaled FP16 SUM. Optional sparse attention. Native single-GPU FP32 audio; conditioning, sampling and public VAE planning retained.

UPDATED 22 OCTOBER 2026 | 46 PAGES

Pinned ComfyUI: d49e888586dd8ae012c0667b33466b815fee07f7

Repository: 2xT4 minimax h3

C:/Users/Administrator/Documents/project/2xT4 minimax h3

Real T4 operator results; full-pipeline quality and latency remain unqualified.

# How to read this guide

A source-first explanation of what runs, why it helps, and what is still unproven.

The target is a two-rank MiniMax-H3 inference service for two T4 GPUs. The optimization is a combination of numerical operators, model partitioning, bounded weight movement and lifecycle control. Faster kernels alone do not explain its behavior.

| Read in this order | Questions answered |
|---|---|
| Architecture and policy - pp. 3-5 | Who owns the work? Which choices are defaults? |
| Attention - pp. 6-10 | What is quantized? How do dropping and pooling differ? |
| Linear operators and LoRA - pp. 11-17 | Why global scales and INT32 SUM? Where are buffers removed? |
| Loading and VAEs - pp. 18-25 | How do disk, CPU, GPU windows and decode stages cooperate? |
| Preservation and validation - pp. 26-28 | Which evidence applies? How should the system be operated? |
| Integration audit - pp. 29-37 | Which supporting optimizations and historical decisions matter? |
| Current defaults / evidence - pp. 38-42 | What was promoted, measured and still unqualified? |
| Source catalogue - pp. 43-46 | Where can mechanisms and measurements be checked? |

## Three distinct meanings of correctness

**Same-method conformance** means an optimized implementation matches its own numerical reference. **Native equivalence** means it matches the selected original operator at a defined boundary. **Generation quality** means trajectories, content, temporal structure and audio remain acceptable. Passing one does not imply the others.

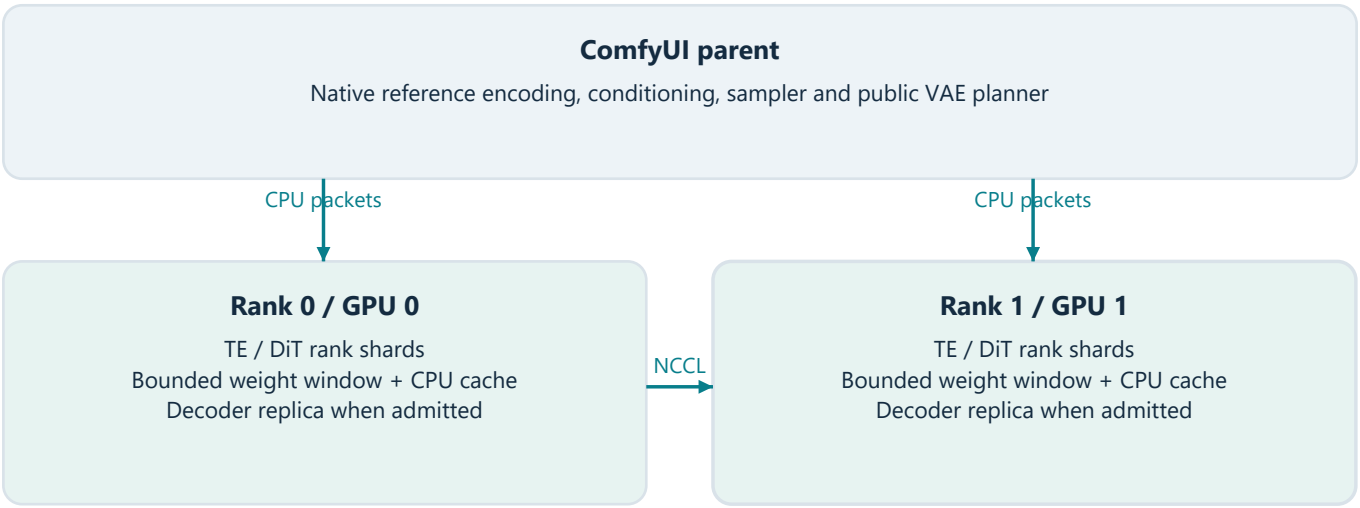
> **Reading the figures**
> Teal marks a current default or active work. Amber marks an optional approximation or policy. Grey marks Native behavior, preserved control flow or inactive storage. Diagrams are explanatory schematics unless a chart explicitly names a measurement.

**Notation:** S is the complete packed token count, M the flattened row count, K the linear input width, N the linear output width, D the attention head dimension, and r a TP rank. MiB = 2^20 bytes; GiB = 2^30 bytes. Kernel time, operator time and request time have different boundaries.

Source: S01, S03, S30, S33-S39

# One Native pipeline, two persistent ranks

Keep ComfyUI semantics in the parent; distribute the expensive model work.




One session lock and an explicit phase protocol coordinate both owners.

The ComfyUI parent retains prompt/image/audio preprocessing, public model interfaces, sampler scheduling and public VAE planning. A session owns two persistent workers, each bound to one CUDA device. Weights can remain in bounded CPU caches while GPU weight windows change between TE, DiT and decode phases.

| Boundary | Ownership and consequence |
|---|---|
| Parent to ranks | Bounded synchronous CPU packets preserve tensor dtypes, nested containers and PackedLayout metadata. Persistence amortizes startup; this is not an asynchronous transport-overlap claim. |
| Between ranks | Actual distributed collectives complete the TP row projections and VAE exchanges. The workers share the same configuration and collective order. |
| Phase transition | TE, sampling and VAE decode have explicit lifetimes. Incompatible phases release or fence their owners before another stage is admitted. |

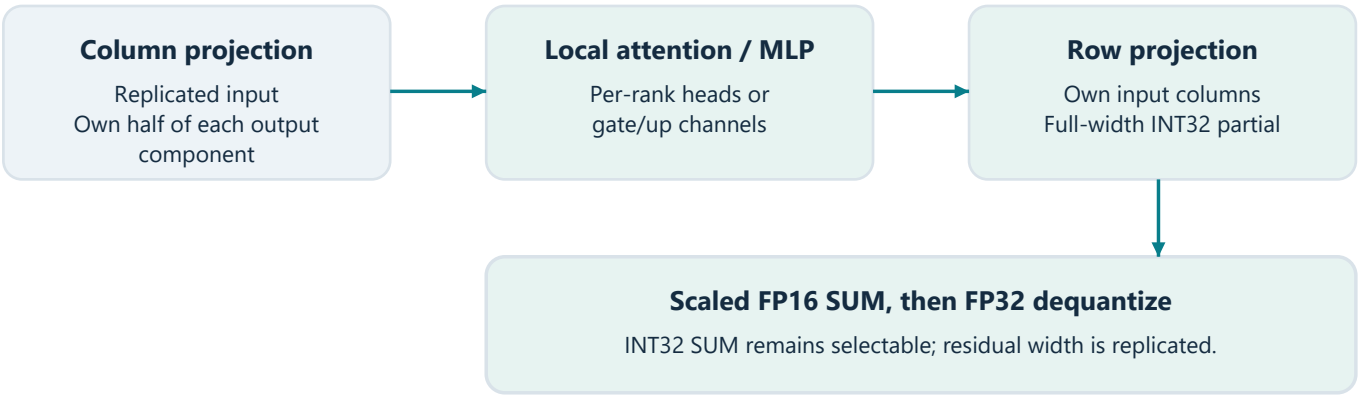
Workers set NCCL_P2P_DISABLE=1 and both minimum/maximum channel counts to 4, leaving NCCL protocol selection at its default. Direct GPU peer copies are not assumed. Those settings are a runtime contract, not a measurement of physical PCIe bandwidth.

> **Important limit**
> One-GPU logical-rank previews test arithmetic and wiring. They do not validate physical NCCL behavior, two-T4 latency or concurrent-request throughput.

Source: S03-S05, S23-S26

# TP2 changes the shapes that matter

Partition each packed component correctly, then recombine row-projection sums.




| Per-rank quantity | TE / Qwen3-VL | DiT main blocks |
|---|---|---|
| Hidden / output width | 5120 | 5376 |
| Attention heads; D | 32 Q, 4 KV; D=128 | 28 Q/K/V; D=128 |
| Local Q or attention width | 4096 | 3584 |
| Local FFN activation width | 12800 | 7168 |
| Gate/up representation | Separate tensors | Packed local width 14336 |
| Row O: local K -> N | 4096 -> 5120 | 3584 -> 5376 |
| Row FFN: local K -> N | 12800 -> 5120 | 7168 -> 5376 |

Column-parallel projections split output rows. DiT QKV is three components, and FC1 is two: each rank owns half of each component, not a single half of the concatenated tensor. Row-parallel projections split the input dimension and leave the full output width on both ranks.

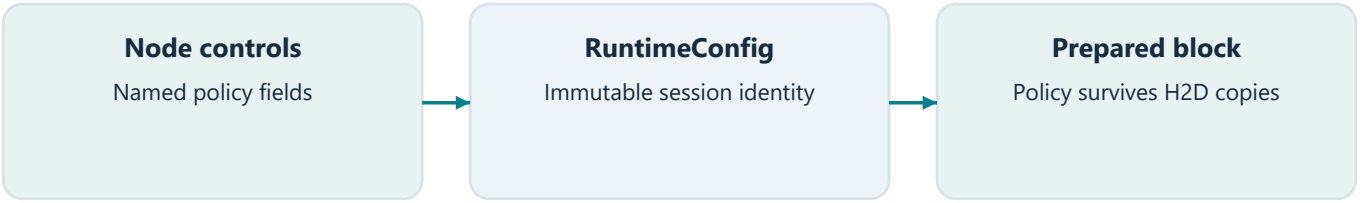
For a row projection, each rank computes a local INT32 partial P[r]. The output is reconstructed from SUM(P[r]) using the shared activation scale and output-channel weight scale. RMSNorm and DiT AdaLN are replicated. Sequence lengths vary with Native packing; a historical S=6814 fixture is not a fixed deployment contract.

Source: S06, S07, S16, S21

# A per-session operator policy

Backend selection is explicit and becomes part of cache and worker identity.

| Setting | Current default | Alternative / scope |
|---|---|---|
| TE attention | tilelang_group32 | native |
| DiT main attention | kitchen | tilelang_dense / tilelang_drop / tilelang_scalar / native |
| Token Refiner | Native SDPA | Independent of DiT main attention mode |
| Row-linear backend | tilelang_scaled_fp16 | tilelang_packed INT32 / native |
| FC2 quantizer | fused_stored | fused_recompute / owned_fp32 / native |
| SwiGLU padding / selective B | True / True | Each has eligibility guards |
| Scalar score cap | 96 MiB per rank | 0 selects inline scalar summaries |
| LoRA loading / row A | online / FP32 partial SUM | offline / full-input gather |
| Adaptive lookahead / idle | False / False | Explicit runtime options |
| VAE decode | Video auto; audio native | Video native override |




Different policy -> different service/cache identity; no stale backend reuse.

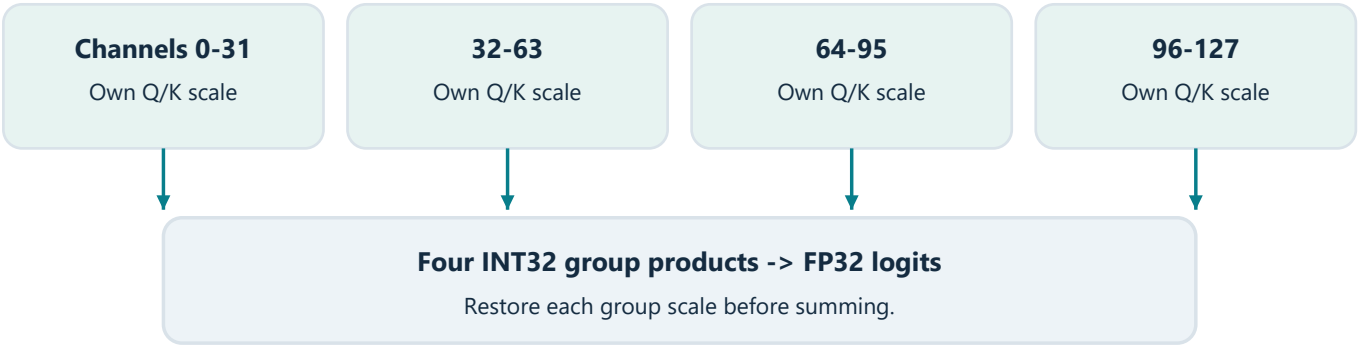
The loader constructs an immutable H3OperatorPolicy and RuntimeConfig. Configuration equality decides whether the persistent service may be reused. Operator identity also follows prepared blocks, residency copies and idle-prefetch cache keys; changing a backend must not reuse a block configured for another backend.

**Do not confuse two settings**
RuntimeConfig retains native_int32 for base/fallback operators. H3OperatorPolicy now selects scaled FP16 row SUM by default. fused_stored prepares TE down and DiT FC2 in FP32 before INT8 GEMM; LoRA is added in FP32 before one FP16 store.

Source: S01-S03, S13, S14, S18

# What Group32 INT8 attention computes

Fine-grained Q/K scales, an INT32 dot product, and an FP16 probability-value path.




For each token and each 32-channel group g, a scale is computed from the absolute maximum. Q is quantized directly. K is centered first. With D=128, the dot product consists of four 32-channel INT8 products, each restored with its own pair of FP32 scales.

```
sQ[i,g] = max(abs(Q[i,g])) / 127, floored at 1e-8
QI[i,g] = clamp(round(Q[i,g] / sQ[i,g]), -127, 127)
logit[i,j] = sum_g INT32(QI[i,g] dot KI[j,g])
             * sQ[i,g] * sK[j,g] / sqrt(D)
```

K centering uses 64-key leaf centroids and the unweighted mean of those centroids, including the ragged final leaf with its actual count. Subtracting one common K offset changes every logit in a query row by a common constant in exact arithmetic, so softmax is invariant. Quantization and finite-precision boundaries still introduce approximation.

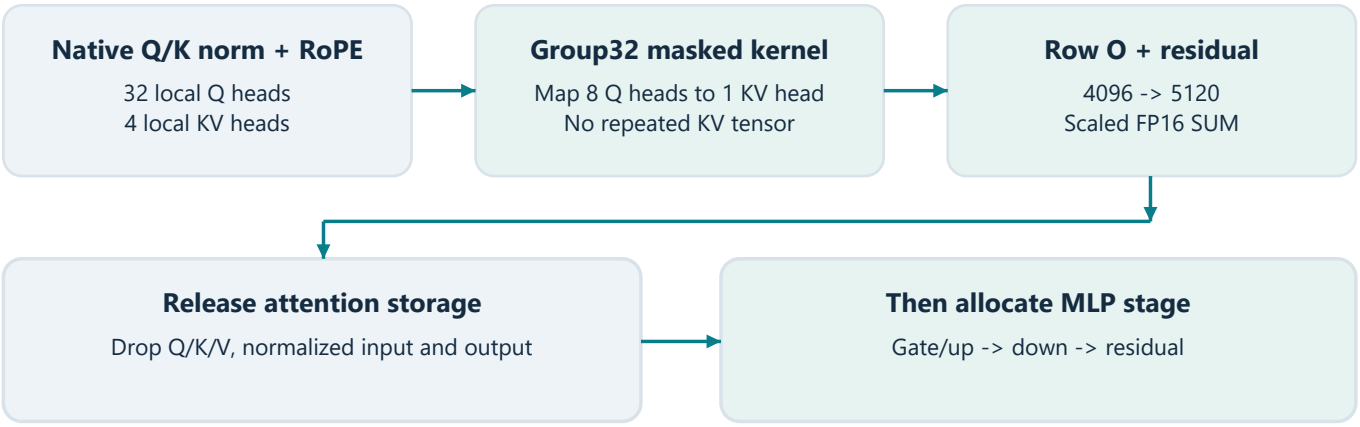
Online softmax maintains a running maximum, denominator and weighted numerator as each 64-key leaf arrives. It avoids allocating a full S-by-S score tensor. Softmax state and output accumulation use FP32; probabilities and V feed an FP16 PV GEMM. The input preparation retains the selected FP16 rounding boundary.

> **Precision meaning**
> Group32 offers a more local Q/K representation than coarser scaling, but this document does not claim universal accuracy superiority over Kitchen. FP16 PV is the integrated TileLang path; the experimental INT8-PV attention variants are not enabled here.

Source: S08-S10, S33

# TE: compact GQA, real masks, short lifetimes

Preserve Native embeddings, Q/K normalization and RoPE before the attention boundary.




Each rank has 32 Q heads but only 4 K/V heads. A query head maps to its shared KV head inside the TileLang kernel, avoiding explicit replication of KV storage. This is grouped-query attention (GQA), not a change to the learned projections.

| Contract | Implementation behavior |
|---|---|
| Mask shapes | 2D, 3D and 4D masks must broadcast to the local BHND query/key shape. Boolean masks become additive 0/-infinity values on device. |
| Online softmax | Masks enter the exp2-domain logits. A fully masked query returns zero. A masked early leaf does not prevent later valid leaves from contributing. |
| Native fallback | Unsupported architecture logs a fallback. Native backend selection preserves the full-model size policy, including short-prompt masked GQA behavior. |
| Scope | TE prefill only. Past-KV-cache execution remains rejected. |

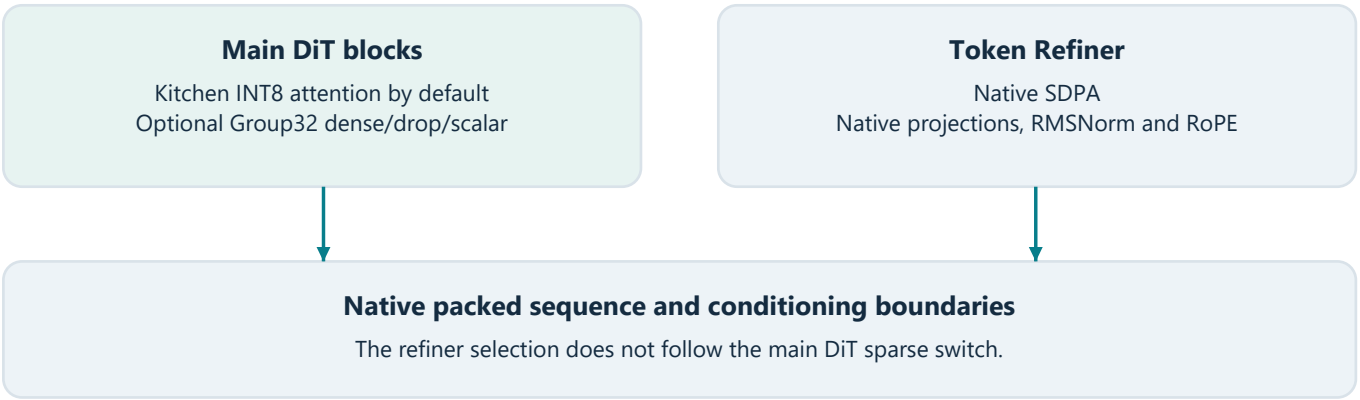
After the attention residual accumulation, normed input, Q/K/V and attention output references are deleted before allocating MLP intermediates. After down projection, gate/up/normalized inputs are released before the MLP residual, followed by its output. This changes reference lifetimes, not residual arithmetic order.

> **Why explicit deletion helps**
> A view or closure can keep an entire storage alive. Removing only one alias is insufficient. In inference, releasing the last Python owner permits allocator reuse after stream safety is satisfied; autograd outside this runtime can retain its own references.

Source: S06, S08, S34

# Kitchen attention is the main DiT default

The Token Refiner deliberately retains Native SDPA.




Main DiT blocks normalize and modulate their inputs, project local Q/K/V, apply Native Q/K norms and RoPE, then dispatch through the operator policy. The default calls comfy_kitchen.int8_attention. Its result is converted to token-major local features before the row-parallel O projection and residual update.

Kitchen and TileLang Group32 are different numerical methods. Kitchen remains the selected DiT and Video VAE attention implementation; the optional TileLang dense path uses the Group32 QK / FP16 PV contract. Historical experiments with alternative PV precision or centering do not silently change this selection.

## The refiner is a separate boundary

NativeRefinerAttention wraps the existing refiner module rather than storing bound methods that could still point at canonical CPU weights after residency copying. It keeps the Native projections, RMSNorm and fused RMSNorm/RoPE behavior, and explicitly calls PyTorch scaled_dot_product_attention. Refiner tokens are never routed through sparse DiT attention.

> **An old name is not an old execution path**
> KitchenRefinerAttention remains only as an import-compatible alias for the Native SDPA wrapper. It no longer selects INT8 attention. This deliberate change is not byte equivalence with the former INT8 refiner.

For the optional dense Group32 DiT kernel, the current caller uses a 64-query tile, 128 threads, a row warp policy and cached Q operands. Such launch/resource choices are architecture-specific implementation details, not independent guarantees of Tensor Core utilization or pipeline speedup.

Source: S01, S07, S09, S33, S34

# Which DiT attention blocks may be omitted?

Route 64-token leaves; protect conditioning through the current Native layout.

Not a captured route map.

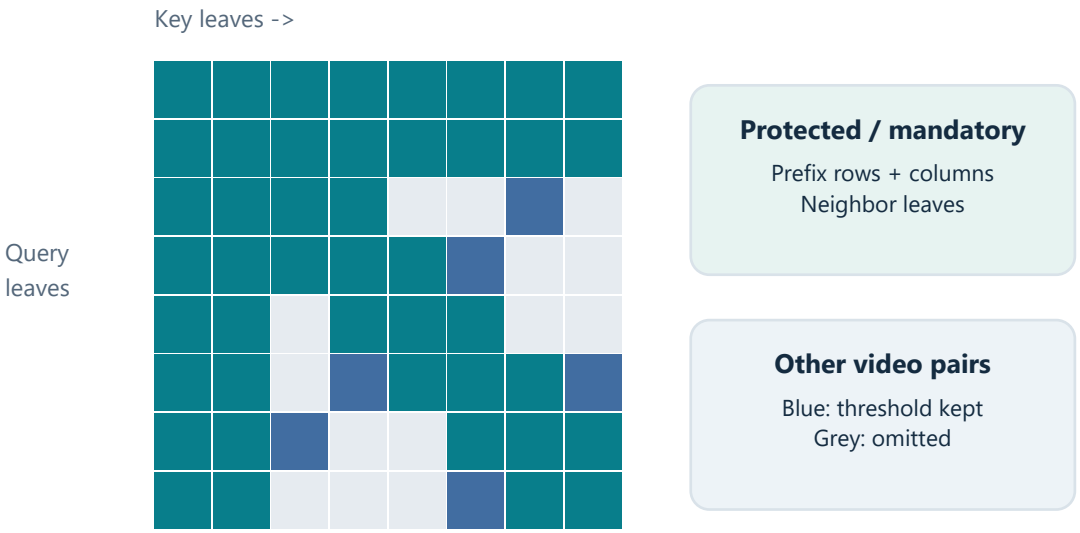



Illustration: 2 prefix leaves + 6 generated-video leaves

The sparse modes require DiT self-attention with matching Q/K/V shapes. Query and key tokens are grouped into leaves of 64. A query-leaf centroid is compared with each centered key-leaf centroid; tau0 retains scores above the query-dependent mean threshold. Nearby leaves and the first key leaf are mandatory.

The runtime obtains the dense prefix from the current PackedLayout. All text, references, conditioning and generated audio before the final generated-video segment remain protected. A missing or stale layout is rejected, not replaced with the historical 874-token fixture constant. Protection rounds to whole leaves, so a leaf intersecting the prefix is retained as a whole.

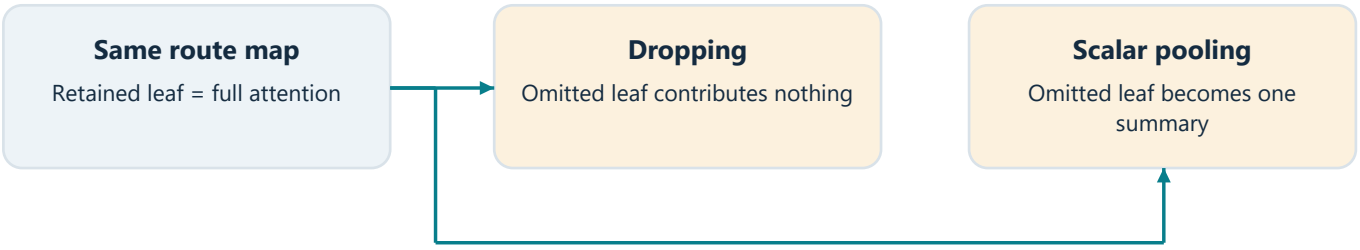
| Always retained | Policy consequence |
|---|---|
| Prefix query leaves | Attend all key leaves. |
| Prefix key leaves | Remain visible to all query leaves. |
| Local neighborhood / first key leaf | Provide the routing method's mandatory connectivity. |
| Other video-video pairs | Centroid threshold decides whether full token work is kept. |

**Sparsity is not speedup**
Routing, preparation, launch overhead and work on retained blocks remain. Scalar pooling also processes omitted-leaf summaries. A percentage of omitted blocks is not a measured percentage of request-time savings.

Source: S01, S10, S11, S38

# Dropping and scalar pooling are different

Both are optional zero-shot approximations; retain both choices.




Pooling preserves approximate leaf mass, not token-level attention structure.

| For an omitted leaf | Dropping | Scalar pooling |
|---|---|---|
| Logits / denominator | No contribution | One centroid logit plus log(actual token count) |
| Value contribution | No contribution | Weighted mean V of the omitted leaf |
| Retained leaf | Full Group32 QK and FP16 PV | Same full Group32 QK and FP16 PV |
| What is lost | All omitted probability mass | Within-leaf logit/value variation and correlation |

```
pooled_logit[i,b] = dot(Q[i], K_centroid[b]) / sqrt(D)
                  + log(count[b])
pooled_numerator += exp(pooled_logit - running_max)
                  * V_centroid[b]
```

The log-count term represents multiple tokens with one summary; simply adding one unweighted centroid token would change the intended mass. For a ragged leaf, count is the true number of keys. The FP16 pooled-probability boundary and streaming normalization remain part of this approximation.

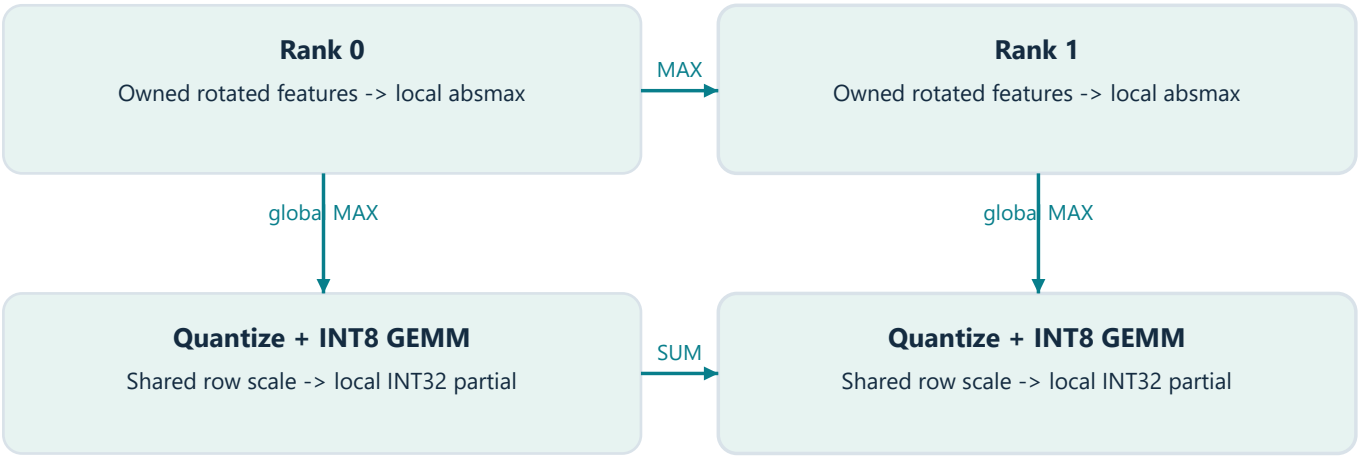
Scalar mode may precompute only omitted-leaf query-centroid scores. The workspace is B*H*Np*Nb*4 bytes per rank. A retained real H3 fixture requires 82,066,432 bytes (78.3 MiB), below the 96 MiB default cap. A zero cap or a larger required workspace selects inline summaries. This trades one preparation kernel and memory for less centroid work in the main kernel.

> **Quality boundary**
> Sparse implementation conformance is checked against the same routing/pooling method. Sparse-versus-dense L1/L2 is not a generation-quality gate. Semantic/distribution drift and temporal behavior require full-pipeline generation review, which is outside these integrated operator checks.

Source: S01, S10, S11, S38

# Global row scales make TP INT8 coherent

Local integer GEMMs are only valid together with the right cross-rank contract.




Prior packed/INT32 control shown; current default uses scaled FP16 SUM.

```
m[i] = MAX_r max(abs(rotated_X[r][i,:]))
sA[i] = max(m[i] * float32(1/127), 1e-30)
Q8[r] = clamp(nearbyint(rotated_X[r] / sA), -128, 127)
P[r] = Q8[r] @ W8[r].T                  # INT32
Y[i,j] = float32(SUM_r P[r][i,j]) * sA[i] * sW[j]
```

The maximum must cover the entire logical activation row, even though each rank owns only part of it. Independent half-row scales would make the integer partials incompatible. Per-output weight scales also retain the full-width checkpoint quantization semantics; they are not recomputed independently on input-column shards.

The packed row path optimizes ownership padding and activation preparation. It still uses Kitchen's native INT32-accumulating INT8 GEMM and an INT32 AllReduce SUM before dequantization. Here, packed does not mean compressed INT8 network transport.

ConvRot groups contain 256 channels and are fully owned locally. The row quantizer follows its own Kitchen-compatible floor, fast division, nearby rounding and [-128,127] saturation. This is distinct from attention's Group32 quantizer with floor 1e-8 and [-127,127] range; interchanging them would change the numerical method.

Source: S12, S13, S16, S17, S21

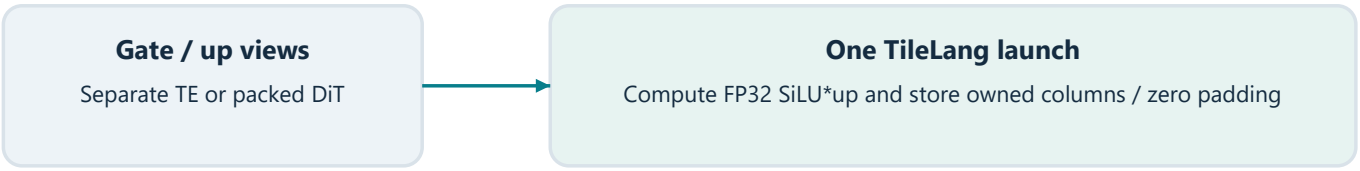
# Fuse SwiGLU with ownership padding

One output write replaces eager activation and padding stages.

Before / full-width preparation




Fused path




SwiGLU computes silu(gate)*up. TE supplies separate gate and up tensors; DiT supplies strided halves of one packed FC1 result. The fused TileLang kernel reads either layout directly. It writes the owning rank's output columns and explicitly writes positive zero into unowned columns.

The full-width mode feeds the same Native FP32 ConvRot GEMM geometry. It preserves FP32 exp, rounded addition/division and multiplication; it does not substitute a fast approximate sigmoid, FP16 activation or a different row quantizer. With eligible shapes, one launch replaces eager SwiGLU plus padding work and removes the separate activated local tensor.

| Site | Input / local activation | Padded output |
|---|---|---|
| TE down | Separate gate/up, each M x 12800 | M x 25600 FP32 |
| DiT FC2, full-width mode | Packed M x 14336; local activation 7168 | M x 14336 FP32 |
| DiT FC2, owned mode | Same packed input; local activation 7168 | M x 7168 FP32, no global padding |

**Fusion boundary**
This removes intermediate traffic and launches around activation preparation. It does not fuse the INT8 GEMM, MAX collective, SUM collective and residual into a single kernel. Complete-row and full-request effects must be measured at their own boundaries.

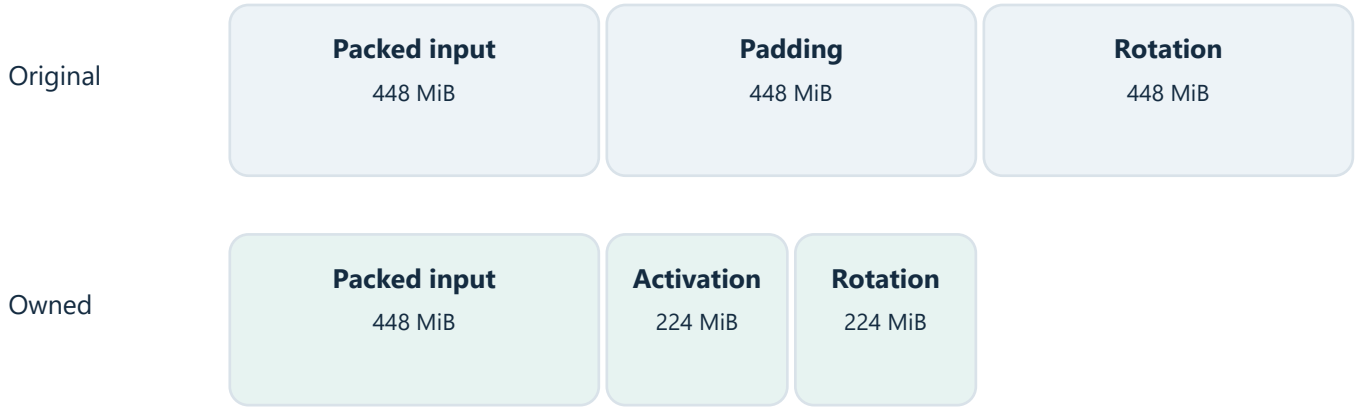
The shared helper also releases the full padded tensor in the actual caller after its last rotation read. An inner helper's del cannot free a buffer still referenced by the caller. Same-stream last-use ordering is retained.

Source: S12, S13, S30, S34

# Rotate only the channels the rank owns

The prior owned_fp32 alternative removes full-width FP32 temporaries.

Illustrated live payload at S8192, per rank (not total VRAM)




At quantization the original also retains 56 MiB INT8 + row vectors.

At S8192, each rank's packed FC1 output is 448 MiB. The original path overlaps it with 448 MiB of zero padding and 448 MiB of full-width rotation. Padding was also retained through quantization, where the owned INT8 output adds 56 MiB. Repairing lifetime alone reduces that latter overlap.

Owned FP32 rotation materializes only the local 7168-channel SwiGLU activation (224 MiB) and its rotation (224 MiB). All local 256-channel ConvRot groups are complete. The same global FP32 MAX, INT8 rounding, learned weight scales and INT32 SUM remain. No lossy activation conversion or compressed transport is introduced.

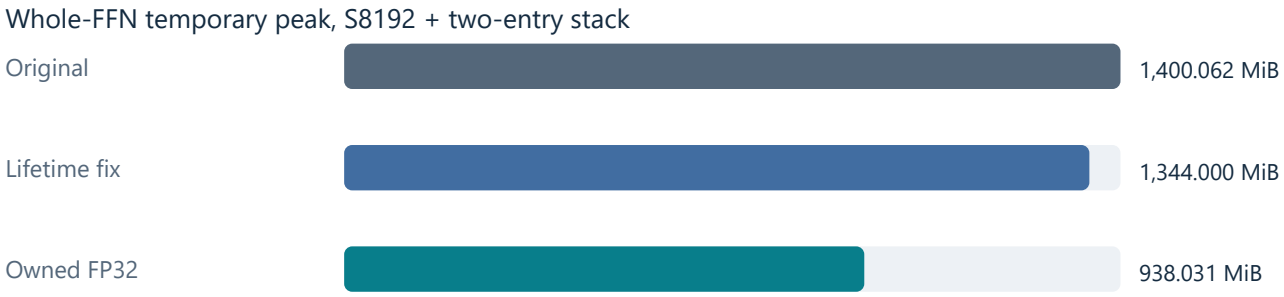
> **The arithmetic caveat**
> The flattened M dimension of the FP32 rotation GEMM changes. Algebraic group locality alone cannot prove identical rounding across algorithms/backends. The trained qualification matrix observed identical payload, scale bytes and complete FC2 outputs; this is not a universal bit-exactness theorem.

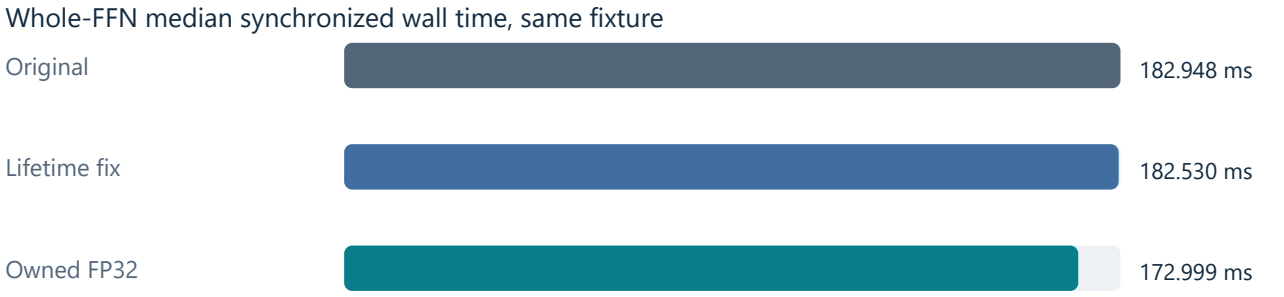
Eligibility is deliberately narrow: packed H3 FC2 width 14336, FP32, TP2, group size 256, packed row backend and enabled SwiGLU fusion. Other shapes/backends retain their existing path. TE down keeps the full-width rotation with the lifetime repair. Set fc2_quantizer=native to restore full-width geometry.

Source: S01, S13, S30-S32

# FC2: memory reduction survives integration

Actual historical two-T4 operator measurements, not an end-to-end forecast.




The charts use S8192 with the retained two-entry LoRA configuration and its existing optional Row Partial mode. Fourteen synchronized wall samples span two reversed rounds; the slower rank is selected per iteration. Temporary memory is measured above the resident entry in separate peak passes. It excludes allocator reservations and NCCL internal allocations.

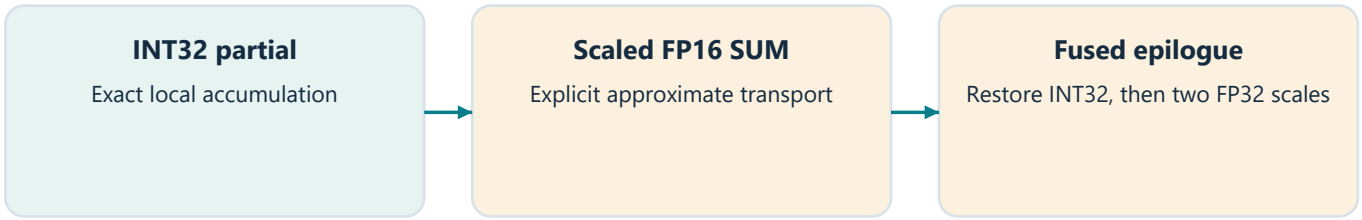
The owned path reduces whole-FFN temporary peak by 33.00% and the pooled FFN median by 5.44% in that allocation. The peak moves to unchanged base FC1 dispatch, whose two output buffers plus INT8 activation/scales total 938.031 MiB. FC2 itself is no longer the largest explicit FFN boundary.

**Scope and version**
These physical results used the historical T4 software stack. They do not qualify the current frozen conda stack on Linux. S32768 integrated confirmation also passes byte gates, but its two-entry whole-FFN timing varies by round: -0.56% then +5.02% reduction. Do not advertise its pooled gain as stable.

Source: S30-S32, S29, S39

# Scaled FP16 SUM with a fused epilogue

The promoted default trades communication precision; exact INT32 SUM remains selectable.




**Default: scaled FP16 SUM**

Exact INT32 SUM remains selectable; representation changes the arithmetic.

```
shift = max(0, ceil(log2(2*Klocal*127*127 / 60000)))
payload[r] = FP16(float32(P[r]) * 2**(-shift))
reduced = AllReduce_SUM(payload)
restored = INT32(float32(reduced) * 2**shift)
Y = (float32(restored) * activation_scale) * weight_scale
```

The power-of-two factor uses a worst-case bound and adds no extra MAX collective. FP16 halves the logical per-rank SUM payload from four to two bytes per element. That is payload accounting, not a guarantee of half the wire traffic, collective duration or total operator latency.

The TileLang epilogue reconstructs the specified INT32 value and performs the two left-associated FP32 scale products in one kernel, avoiding a separate reconstructed INT32 allocation. The policy promotes FP16 storage to FP32 preparation; fused SwiGLU consumes FP16 producers directly. Transport requires TP2, aligned local groups, INT8 weights and contiguous vector FP32 weight scales on physical SM75. Unsupported contracts fall back before collectives.

> **Cancellation makes this approximate**
> Let P0=2^shift*2049 and P1=-2^shift*2048. The exact sum is 2^shift; FP16 encoding can round both magnitudes to 2048, producing zero. Agreement with an independent FP16 codec reference proves implementation correctness, not equality to Native INT32 SUM or preservation of generation semantics.

Source: S01, S14-S16, S34, S38

# Selective B removes unowned FC1 outputs

Keep adapter order, separate GEMMs, and post-B scaling.




B output row order is [gate | up], not [all rank-0 | all rank-1].

Online LoRA adds a floating-point bypass around the INT8 base projection. For each adapter, hidden=linear(x,A), delta=linear(hidden,B), then multiply by its effective strength and add to the target. Adapters execute in Native order. Strength is not folded into B, and A factors are not concatenated into a different GEMM.

Selective B is enabled only for qualified main DiT FC1 geometry: A=[128,5376], B=[28672,128]. Each rank needs 7168 gate rows and 7168 up rows. Those rows are read and packed once on CPU, so B produces 14336 owned outputs rather than all 28672. Unqualified shapes, QKV, refiner sites and other targets keep full B behavior.

| LoRA choice | Semantics / status |
|---|---|
| Selective FC1 B, default on | Skips unowned B rows for the guarded geometry; validated selected-output arithmetic, not a general GEMM-rounding theorem. |
| Native row-input gather, optional | Assemble the complete input before A, retaining Native A GEMM geometry. |
| Row A partial reduction, default | Compute owned A projections and FP32 SUM the small hidden. Changes Native reduction grouping; full-input gathering remains selectable. |
| Concat A | Retired; requesting it raises a clear error. |

Row Partial staging is bounded to 32 MiB of small hidden tensors and only used when this can replace a larger SwiGLU activation. Neither selected B nor bounded staging changes the base INT8 GEMM or enables approximate SUM for that base projection.

Source: S01-S03, S18, S30, S33

# Bake whole layers across two GPUs

Move repeated inference adapter work to a reusable checkpoint conversion.




Offline mode first releases the previous TP service. A size-balanced assignment gives each affected DiT/refiner layer to one of up to two spawned GPU workers. Each worker processes one full-width weight matrix at a time; a matrix and its quantization scale are never split between GPUs.

For an INT8 target, the worker performs Native FP32 dequantization, ordered adapter merging, and deterministic full-input-width ConvRot requantization. Dense targets return to their original storage dtype. Quantizing the full width before later TP splitting is essential: independent half-width weight maxima would change the model.

The base checkpoint is copied once to a new scratch artifact. Workers write disjoint weight/scale ranges through independent descriptors and fsync their writes. The parent publishes only after all workers finish, with source/stack identities, output signature, assignments and checksums in the receipt. No merged weight tensors travel over NCCL or parent pipes.

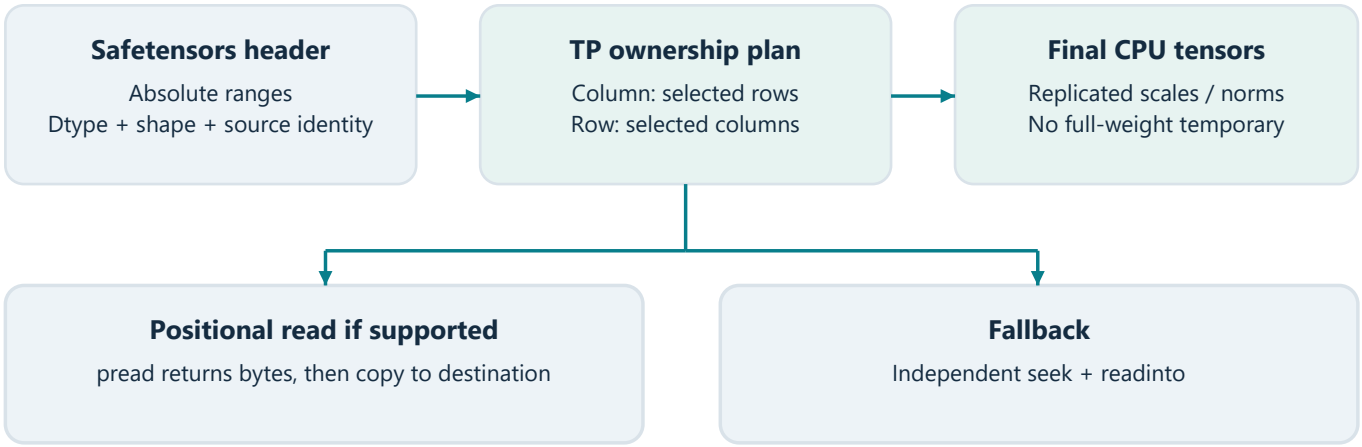
> **Warm reuse and numerical meaning**
> The cache key binds the base and ordered adapter stack, including strengths. A valid baked checkpoint is reused and the online stack is cleared to prevent double application. Baking introduces a new requantization boundary; it is not byte-exact with online LoRA over the original INT8 weights.

Cancellation is checked during copying and between matrices. Failure removes only unpublished scratch owned by this invocation. A new bake needs space for another full diffusion checkpoint. Baking/loading costs are one-time preparation costs, not steady-state inference latency.

Source: S02, S18, S19, S35

# Read owned ranges, not full temporary weights

CheckpointReader supports positional access without changing the model schema.




The safetensors header maps each tensor to a dtype, shape and byte interval. The TP plan reads owned output rows for column projections, owned input columns for row projections, and replicated scales/norms where required. Current files retain the qualified INT8 ConvRot metadata. Source size and modification-time signatures guard against changing a checkpoint during a planned request.

| Path | What it actually does |
|---|---|
| os.pread, when supported | Read at an absolute byte offset, loop on short reads, and copy returned bytes into the final tensor in chunks up to 8 MiB. It removes shared file-cursor mutation. |
| Seek + readinto fallback | Used when pread is unavailable or a descriptor rejects it, including ordinary Windows operation. Correct destination offsets and range order remain. |
| Rank-local offline export | Optional contiguous per-layer/rank safetensors with Native state hashes and source identity. This changes storage layout, not tensor math. |

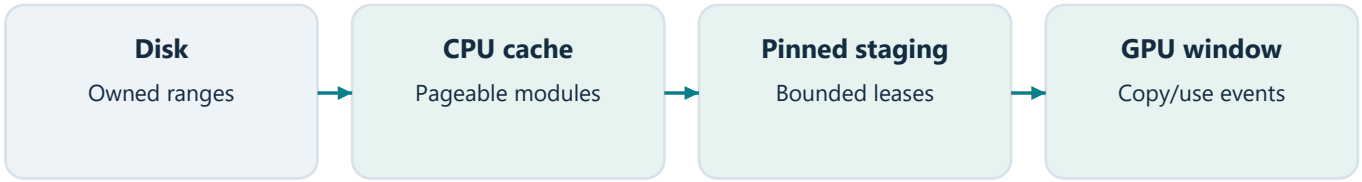
**Pread is not direct I/O**
Python os.pread returns a bytes object; it cannot fill a caller-supplied tensor buffer. This path is neither zero-copy nor O_DIRECT and does not bypass the operating-system page cache. No local SSD result is substituted for T4 storage behavior.

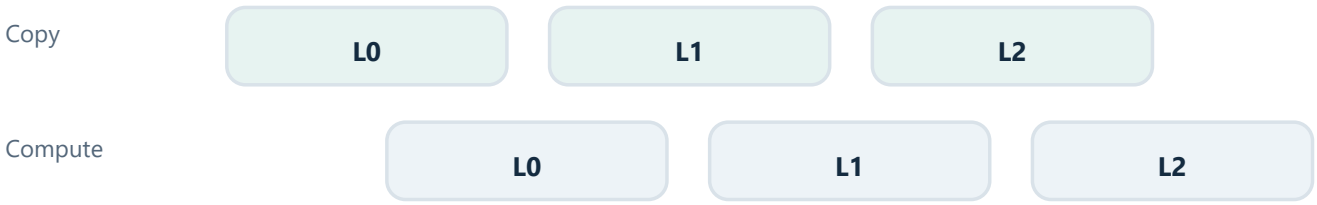
CPU decompression is currently a separate experiment. ZSTD, BitShuffle and FSE benchmarks are not installed in the active loader, so the diagrams do not place a decompressor in the production critical path.

Source: S20-S22, S34

# Bound the pipeline from disk to GPU

Pageable cache, pinned staging and GPU windows have separate owners and limits.




A copy must complete before use; eviction waits for last-use safety.

HostWeightCache prepares immutable CPU modules under a process-shared HostBudget. Per-rank cache capacity limits retained pageable modules, while construction reservations cover preparation peaks. A cache hit can bypass disk/CPU preparation without implying that the module is already pinned or resident on a GPU.

ResidencyWindow admits pin staging, copies modules on a CUDA copy stream, waits on copy events before use, and delays storage retirement until recorded compute reads are safe. Eviction considers future uses and explicit retained keys. CPU and GPU lookahead are scheduling horizons, not permissions to exceed their byte budgets.

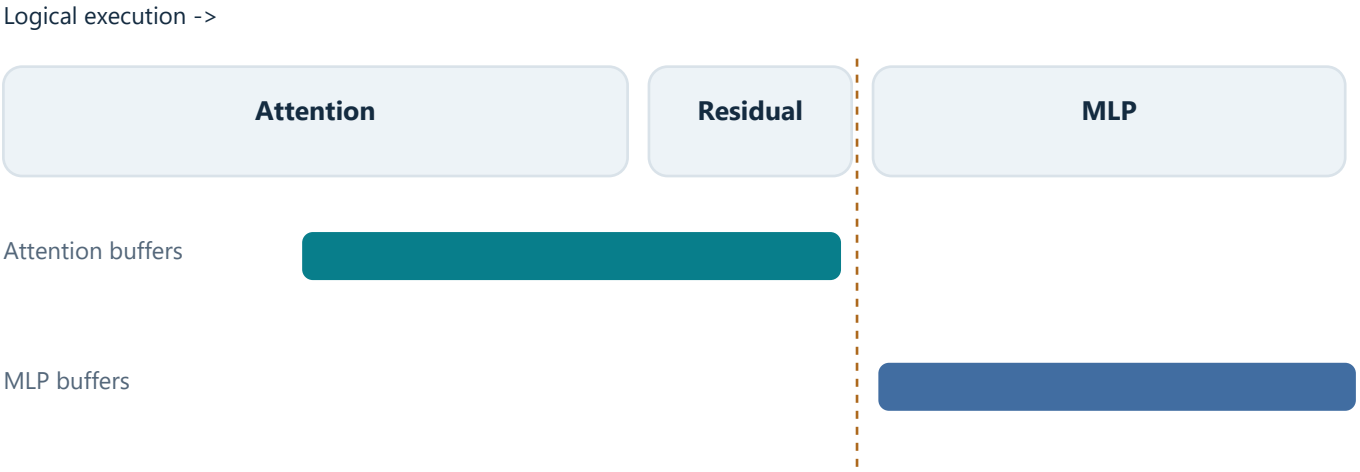
| RuntimeConfig default | Meaning |
|---|---|
| GPU weights: 6 GiB/rank | Live/pending managed weight bound; does not include all activations or library state. |
| Pinned: 6 GiB/rank | Admission bound for staging, with allocator-aware accounting. |
| Pageable cache: 3 GiB/rank | Retained prepared CPU weight cache. |
| Shared host: 16 GiB | Cross-rank managed reservations, not total machine RSS. |
| CPU/GPU horizon: 16 / 2 | Bounded upcoming preparation and distinct GPU groups; reserve remains enforced. |

Adaptive lookahead defaults off. When enabled, the GPU depth becomes one or two upcoming distinct groups based on driver free memory plus reusable allocator memory, the weight cap and reserve. Admission rechecks headroom immediately before every copy. The opt-in also retains the existing CPU timing adaptation.

Source: S03, S20, S22, S23, S34

# Release stages, not just local variables

The last owner determines when a large tensor can leave the live set.




Release last attention owners before creating the MLP live set.

Peak storage is determined by overlap. Summing the largest buffer in each stage overstates a peak if those buffers never coexist; counting only returned tensors understates it when views, closures, caller arguments or autograd retain a backing allocation. The code therefore gives attention, MLP, preprocessing and VAE phases explicit release boundaries.

| Release point | Why it exists |
|---|---|
| TE attention residual | Drop normalized input, Q/K/V and attention output before the MLP starts. |
| DiT attention residual | Release attention views, packed QKV and projection temporaries before FC1/FC2 work. |
| After FC2 rotation | The caller drops padding after its last read; helper-local deletion alone cannot release caller ownership. |
| Encoder to sampling / decoder | Release staged encoder model, patcher and host lease before admitting the next large phase. |
| Close / abort | Fence/release workers, parent leases and idle speculation; unregister closed-service atexit callbacks. |

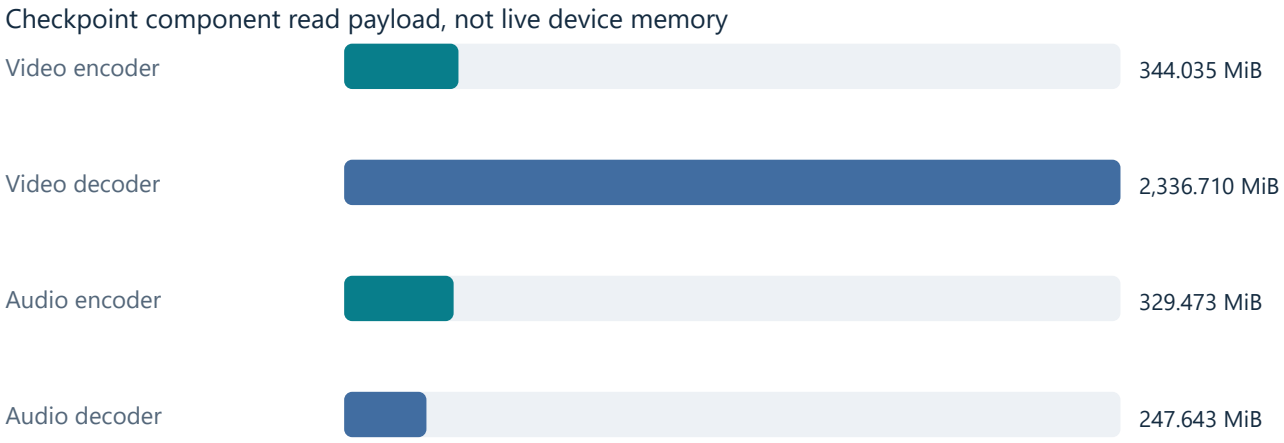
**What memory numbers mean**
Named tensor payload, PyTorch allocated peak, allocator reserved peak, managed host reservation, process RSS and device/NVML usage are different quantities. The FC2 chart reports scoped PyTorch temporary allocation; it is not a 33% claim for full-pipeline VRAM.

The owned-FC2 ledger moves a representative block-created payload maximum to the O projection, where attention buffers remain alive during the row call. Optimizing one stage can expose another bottleneck.

Source: S03, S06, S07, S13, S23, S24, S30

# Load encoders on references, decoders on output

Native public planning remains; the unused model half has no parameter storage.




MiniMaxH3ParallelVAELoader recognizes Native H3 video/audio checkpoint signatures and creates a StagedVAE. A meta-device shell retains Native metadata and public methods. Initial load reads checkpoint metadata only. Encoder weights load on an actual reference encode; decoder weights load on output decode, which may be the first use. Inactive components have no real parameter storage.

Before denoising, registered encoder stages are explicitly released. At first decode, the encoder patcher, model backing and host lease are torn down before decoder reads and construction. A later encode reloads the encoder automatically. The session lock and VAE lock serialize these switches.

| Stage | Retained checkpoint components |
|---|---|
| Video encode / decode | encoder + quant_conv / decoder + post_quant_conv |
| Audio encode / decode | encoder + pre_block + posterior projections / decoder + dec_in_proj |
| Shared / reconstructed | Latent normalization, Native pixel statistics, deterministic nonpersistent decoder RoPE and unit qk_norm_scale buffers. |

**Chart boundary**
Bars show checkpoint payload bytes read for each component, with shared latent buffers included in both stages. They are not peak RAM/VRAM. Parent GPU transfer/offload remains under Native model management; worker decoders use a separate conservative admission.

Source: S02, S03, S24, S35

# Query parallelism keeps global context

Split query ownership, exchange all keys and values, join only final projections.




The video decoder uses all 36 trained transformer blocks. Both ranks hold decoder weights; token/query ownership is split contiguously at a 128-query scale-group boundary. The split includes the five appended register/padding positions. Rank 0 performs Native prefix projection once and broadcasts the other owner's token range. RoPE uses the original global coordinates.

At each attention layer, both ranks project their local Q/K/V. Local Q is kept, while K and V are gathered globally. Each owner therefore attends over the entire key/value context with Kitchen INT8 attention. After the last decoder block, projected outputs are joined and Native patch reconstruction is completed on rank 0.

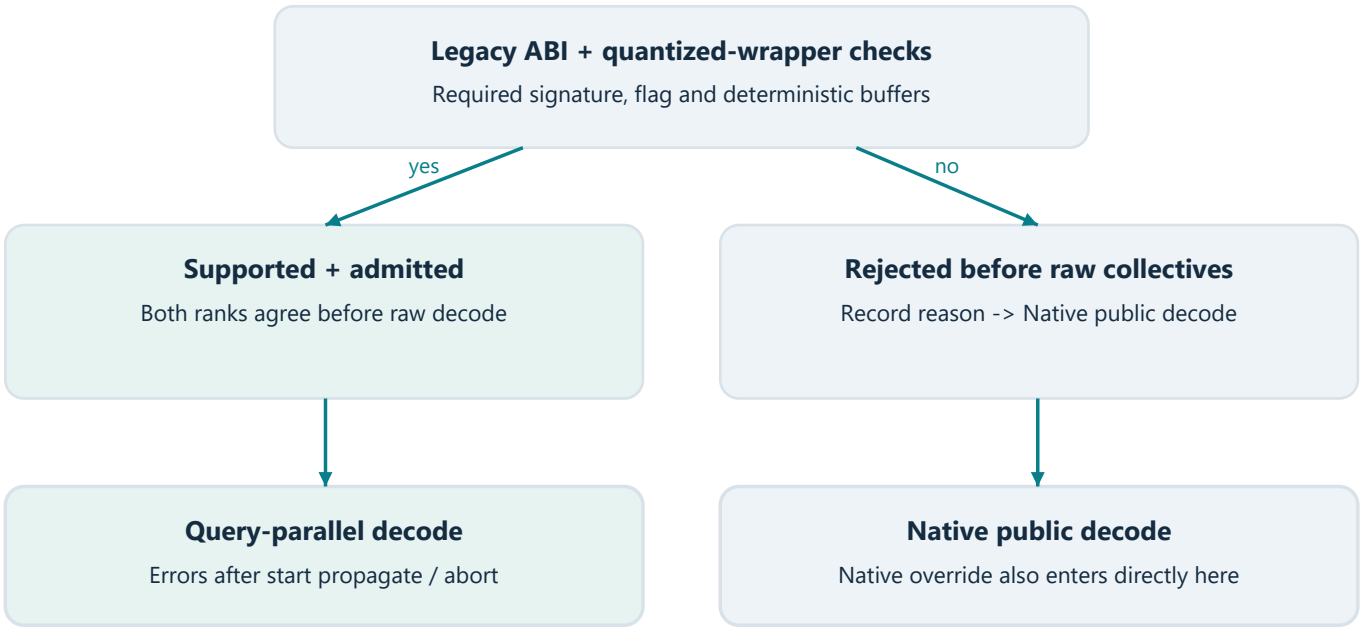
> **Why not independent video slabs?**
> Decoding cropped spatial/temporal pieces independently would remove global attention context and can alter boundaries or consistency. This implementation partitions queries while retaining full KV. It is not head sharding or independent cropped-slab generation.

The parent replaces only the raw _decode_pixels boundary. Native normalization, temporal chunking, spatial tiles, blending, trimming, public output buffers and finalization remain in place. Decoder replicas are reused across raw calls within one public decode and released afterward.

Source: S05, S25, S36

# Compatibility is checked before collectives

The old attention API is preserved because it is part of the qualified contract.




Query partitioning requires the Native Kitchen dispatch flag, quantized QKV wrappers, qk_norm_scale and the legacy attention signature accepting pre_norm, residual and residual_scale. Package import success alone does not prove those contracts. The staged decoder rebuilds nonpersistent buffers that are absent from safetensors.

The parent loads its required stage before VRAM admission so that allocation is visible. Both ranks agree with a MIN collective on conservative host/GPU admission before raw decode starts. Decode mode native bypasses parallel preparation. Very short sequences without two aligned ranges run Native raw decode on rank 0.

> **Two different failure boundaries**
> A compatibility or admission rejection can select Native public decode before raw collectives. An error after collective decode begins must propagate and abort/clean up the service; silently switching one rank to Native could deadlock or accept a partial output.

The sibling ComfyUI-master has a different attention wrapper. The pinned checkout retains the required API. A changed wrapper can disable query parallelism even with the same Kitchen package version. The solution here is source/version preservation and a recorded fallback reason, not an automatic upstream API migration.

Source: S24, S25, S28, S29, S35, S36, S39

# Native single-GPU stereo decode

Audio parallelism is retired; keep the Native FP32 public decode path.

Audio VAE decoding runs on one GPU through the Native stereo decoder. Audio models are not loaded into the two TP workers, no channel split or waveform AllGather runs, and historical auto selections resolve to Native audio decoding.

| Boundary | Current contract |
| --- | --- |
| Input and arithmetic | Native FP32 audio latents, stereo geometry, convolution and decode precision. |
| Public planning | Native chunking, normalization, output layout and model management remain in the parent. |
| Loading lifetime | Load the encoder for reference encoding; load the decoder for output decoding. |
| Historical channel experiment | Retained source/results only. Its geometry-dependent rounding differences are not part of the current default. |

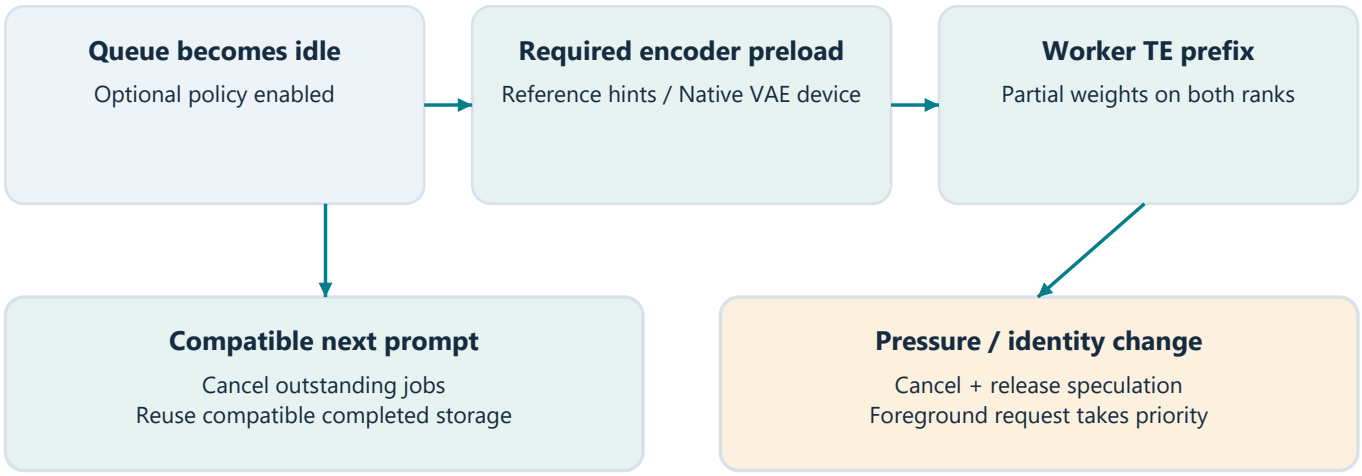
**Numerical boundary**
FP16-fix excludes the Audio VAE. The audio payload still passes through the Native single-GPU FP32 decoder; changes to TE/DiT row transport do not enable audio parallelism.

The Native audio dispatch has an explicit no-worker regression gate. It is not described as an additional dual-GPU speedup.

Source: S03-S05, S24-S26, S36, S58

# Preload encoders, then a bounded TE prefix

Speculation is optional, cancellable and tied to the next compatible request.




No speculative prompt inference or decoder computation is performed.

With idle_residency=True, queue-idle completion preloads only graph-required reference encoders, then asks both workers to begin partial TE preload. Decoder-only VAE links do not select encoders; without a graph, only previously used encoders are selected. Encoders live in the parent on the ordinary Native VAE device, normally GPU 0. They are not replicated on the two TP ranks, and workers see their allocation through driver-memory admission.

Encoder reads check cancellation between tensors. GPU preload needs weight storage plus the configured idle reserve. If GPU headroom is insufficient, a prepared CPU encoder may still be reused. If encoder speculation fails, the reason is recorded and TE speculation may still proceed. No reference encoding, prompt conditioning, initial noise or decoder inference is executed speculatively.

| Event | Lifecycle action |
|---|---|
| New prompt | Cancel outstanding speculation; reuse only compatible completed entries. |
| Reference type changed | Filter encoder requirements by actual reference links; release excluded speculative components. |
| Memory pressure / incompatible identity | Release encoder and TE speculation; foreground work has priority. |
| Denoising starts | Explicitly release staged encoders; active GPU window takes ownership. |

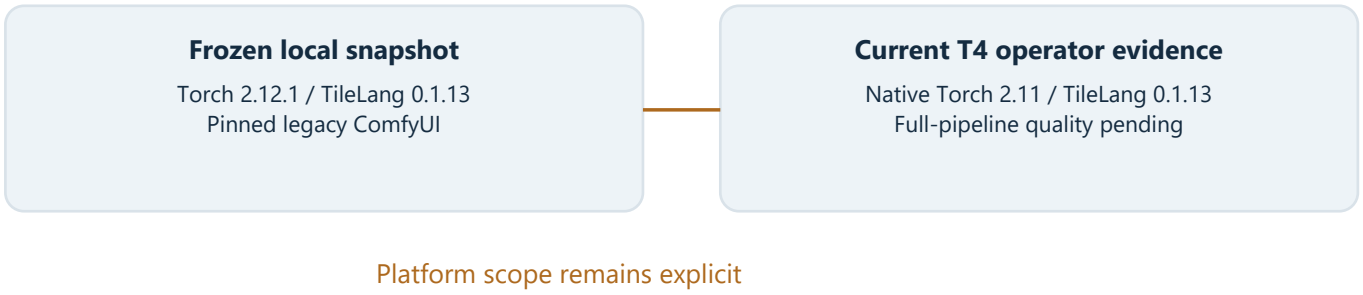
Default idle weight cap is 4 GiB/rank, with a 4 GiB idle reserve and 0.25-second delay; idle remains disabled by default. This is separate from the active 6 GiB managed GPU weight cap. Reuse never bypasses source/adapter/operator identity checks or stream-lifetime fences.

Source: S02, S03, S23, S24, S27

# The validated runtime is an input

Operator adaptation must not silently upgrade source, APIs or dependencies.

| Contract | Preserved local stack |
|---|---|
| ComfyUI source | d49e888586dd8ae012c0667b33466b815fee07f7 |
| Default interpreter | C:/ProgramData/miniconda3/envs/comfyui/python.exe |
| Torch / Kitchen / TileLang | 2.12.1+cu132 / 0.2.37 / 0.1.13 |
| Frozen environment record | runtime/local_conda.json |
| Historical physical T4 operator stack | Torch 2.11.0+cu130, Kitchen 0.2.37, TileLang 0.1.15; nvcc 12.8.93 |
| Current native-Torch T4 operator batch | Torch 2.11.0+cu128 / Kitchen 0.2.37 / TileLang 0.1.13. Full H3 quality remains unqualified. |




The launcher selects the pinned local ComfyUI and checks normalized source identity. It does not replace it with the API-different sibling checkout. The original sibling has no Git identity in the retained provenance record; this document does not claim it was synchronized upstream or rolled back.

Kaggle uses the frozen source and exact core pins with the user-authorized native platform stack. The current 0.1.13 gate follows real T4 operator tests; it does not qualify the Windows Torch/CUDA ABI on Linux. Local conda and original snapshots remain unchanged.

> **No hidden environment switch**
> The historical D:/h3_tp2_legacy_env is not the default. This PDF is generated with the separate bundled document runtime; it does not install or change inference packages. No GPU time is consumed for document creation.

Source: S28, S29, S33, S39

# What the retained checks actually establish

Conformance, physical transport and full-model quality are separate evidence classes.

| Evidence | What it covers | What it does not cover |
|---|---|---|
| Pipeline review: 56 tests | Lifecycle, source/runtime preservation, policy/UI, loading, cache identity, debug isolation | Full H3 generation speed or physical NCCL timing |
| Historical physical integration: 10 gate records/rank | Masked GQA, selected B ownership, drop/scalar dispatch, native INT32 row collectives | Current frozen Linux stack; full-depth quality |
| Enhancements: local arithmetic / contracts | Scaled-FP16 codec reference, refiner SDPA, positional-read fallback, explicit lifetimes | New physical T4 performance for those final sources |
| FC2 physical v1/v2 | Trained owned-rotation payload/scale/output identity; scoped memory/latency; integrated S32768 dispatch | Universal GEMM rounding equivalence or pipeline VRAM |
| Trained VAE local checks | 36-block video query and staged encode/decode agreement in tested fixtures; audio differences recorded | Physical integrated near-capacity two-T4 request performance |

The charts in this guide use retained measurements and do not launch new experiments. Historical sources/results stay version-bound. A test against an approximate reference can pass perfectly while still differing from dense attention or Native INT32 transport.

Nsight Systems evidence in prior operator investigations explains launches, preparation, collective intervals and allocation boundaries. Nsight Compute returned ERR_NVGPUCTRPERM on the retained Kaggle probes. Registers/shared-memory sizes and kernel interval coverage are not substitutes for achieved occupancy, SM utilization, memory-stall or Tensor Core counter measurements.

**Full-pipeline scope**
The complete current default/optional dispatch combinations have not all been qualified for full-depth two-T4 generation, temporal/audio semantics, queued concurrency or end-to-end latency. Operator gains must not be added together into a pipeline speedup.

Source: S30-S39

# Operate, inspect and change one boundary at a time

Use the explicit configuration and preserve Native fallbacks.

```
from patches.comfy_tp_runtime import RuntimeConfig
from patches.h3_operator_policy import H3OperatorPolicy
config = RuntimeConfig(
    te_path, dit_path,
    adaptive_lookahead=False,
    operators=H3OperatorPolicy(
        te_attention="tilelang_group32",
        dit_attention="kitchen",
        row_linear_backend="tilelang_scaled_fp16",
        fc2_quantizer="fused_stored",
        swiglu_padding=True, selective_b=True,
        scalar_score_workspace_mib=96,
    ),
)
```

Use the linked MiniMaxH3TP2Loader and parallel VAE nodes so that all stages share one persistent session. A changed policy or checkpoint/adapter identity recreates the service. Production TileLang dispatch checks SM75; an SM120 arithmetic preview must not be reported as a target run. Physical TP2 requires matching CUDA devices and an available NCCL backend.

| Symptom / investigation | First boundary to inspect |
|---|---|
| Unexpected Native fallback | Selected source identity, GPU capability, VAE attention ABI and explicit decode_mode. |
| High peak memory | Live aliases, parent VAE plus worker replicas, FC1/O stage peaks, weight/pinned/host budgets. |
| No sparse speedup | Preparation/routing/summary costs and complete-API timings; never only omitted-block count. |
| Quality drift | Selected approximation: sparse mode, FP16 SUM, Row Partial or offline requantization. Hold inputs/noise fixed. |

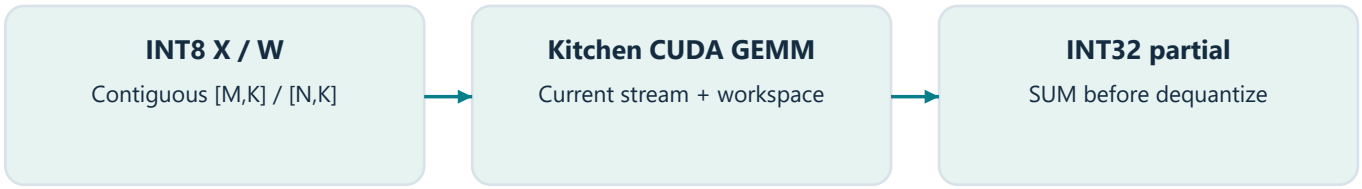
Runtime code is under patches/ and the custom node. Read-only stats snapshots and bounded counters are separate from explicit profilers/captures in benchmarks/. Native reference loaders are lazy qualification paths. Legacy PP/SP/tiled interfaces remain compatible but are not selected by this TP2 loader.

> **Not integrated by this document**
> TE compression codecs, experimental attention INT8 PV, packed-INT8 transport and broad auto-tuning are not enabled. This documentation revision executes no H3 inference, dependency setup job or new profiler capture.

Source: S01-S03, S28, S33-S39

# Use the Turing GEMM storage contract

The integrated base row GEMM avoids the historical transposed-weight and row-padding work.




**Avoid the old per-call layout work**

No M padding or transposed-weight temporary for already qualified contiguous inputs.

Every physical worker installs optimized_tp.enable(scalemax, native_int32) before model execution. This replaces parallel_int8.int_mm with the pinned Kitchen CUDA cuBLAS INT8 entry point. It consumes contiguous [M,K] INT8 activations and [N,K] INT8 weights, writes [M,N] INT32, and uses the current CUDA stream and the backend workspace.

The older torch._int_mm helper padded M to a multiple of 32 and constructed a contiguous transposed weight on every call. The active helper avoids those two temporaries when the inputs already have the required layout. Its contiguous() calls can still copy noncontiguous inputs; this is not a general claim that all GEMM preparation is free.

| Projection boundary | Retained implementation |
|---|---|
| Base/fallback row O / down / FC2 | Original INT8 GEMM and Native INT32 SUM. Default policy overrides eligible rows with scaled FP16 transport. |
| Base column Q/K/V / QKV / FC1 | Native Kitchen int8_linear, with complete input rows and component-aware output ownership. |
| Native quantizer fallback | Keep width/shared-memory guards. A fused route may still require complete rows; fixed equal-width gathering avoids length exchanges and scalar synchronization. Otherwise retain the rotated-matmul route. |
| Native epilogue | Use the CUDA dequantizer with its bias/cast rounding. Eager multiply/add replacements are not assumed equivalent. |

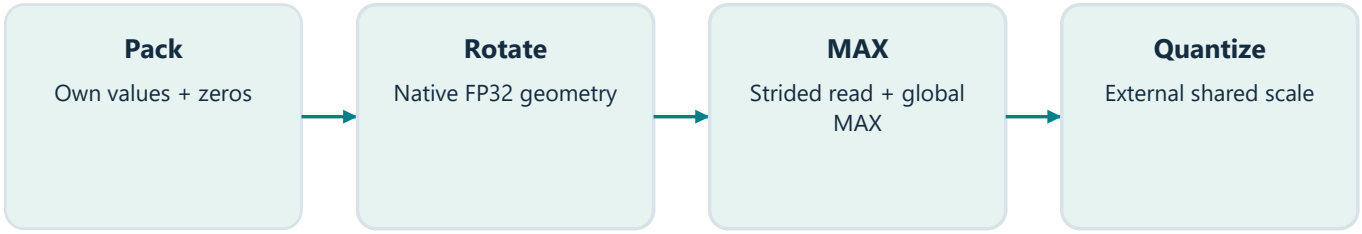
**Evidence and scope**

The historical full-model study attributed TE S512 per-block INT8 GEMM intervals of 8.128 -> 3.882 ms and removal of feature AllGather in the scalemax path. These are diagnostic kernel sums from an older source/stack, not a speed estimate for today's complete pipeline.

Source: S06, S07, S12, S16, S17, S40

# Packed rows combine three accepted changes

External global maxima, strided reads and ownership packing are distinct from SwiGLU fusion.




**Remove staging work around the same row arithmetic**

No sentinel, concatenation, owned FP32 slice or eager abs temporary.

| Stage | Buffer / work removed |
|---|---|
| External-max quantization | Consume the already AllReduced maximum directly; omit the sentinel group, concatenation, redundant row-maximum scan and final INT8 slice copy. |
| Strided owned-slice maximum | Read the owned columns directly from full rotation storage; avoid an owned contiguous FP32 copy and eager abs tensor. |
| Packed padding | Write owned values plus positive zeros in one pass before the unchanged full-width rotation. |
| SwiGLU and owned-FC2 successors | Fuse activation into packing; eligible owned_fp32 FC2 then removes the global-width rotation buffers entirely. |

These are live stages in tilelang_row_quant and the SwiGLU wrapper, not just names from an experiment. The shared maximum still crosses ranks before quantization; the maximum kernel alone does not fuse that collective with the quantizer. Full-width fallback geometry remains where the owned-FC2 guard does not apply.

A controlled fast-division versus rounded-division probe changed 13,382 INT8 elements per rank at constructed half-integer boundaries. The active row quantizer keeps Kitchen's fast division, reciprocal-127 multiplication, 1e-30 floor and nearby rounding. Numerically more accurate division would change the selected method.

**Do not conflate two meanings of packed**

Packed preparation describes tensor ownership and eliminated copies. INT8-packed SUM transport is a separate rejected/nonselected approximation; tilelang_packed communicates INT32; the new default communicates scaled FP16 accumulators.

Source: S01, S12, S13, S16, S17, S41

# Which Kitchen-inspired ideas are actually live?

Prepared operands and direct output stores matter independently of the QK/PV arithmetic.

| Current path | Active implementation detail |
|---|---|
| TE Group32 | BQ32 masked kernel; shared KV-head mapping; fused stride-aware Q/K/V quantize/cast/pad preparation; direct token-major output. |
| Optional DiT dense | BQ64, 128 threads, FullRow warp policy, Q-only reuse; eager K-centering reduction followed by the fused preparation kernel. |
| Optional drop / scalar | BQ64 with fused Q/K quantizers and finite-centroid tau0 shortcut; direct BND store. Drop does not prepare unused V centroids. |
| Geometry cache | Bounded LRU caches reuse immutable counts/dummy buffers and compiled specializations; CUDA stream identity protects asynchronously initialized geometry. |
| Output boundary | Write BND in the requested dtype directly, while explicitly preserving the intermediate FP16 output rounding. |

Q-only reuse retains query operands/scales across the key-leaf loop without caching every K group. Direct BND avoids the separate transpose/reshape-copy and dtype-conversion path at the integration boundary. Such savings do not change the leaf64 softmax order or turn FP16 PV into INT8 PV.

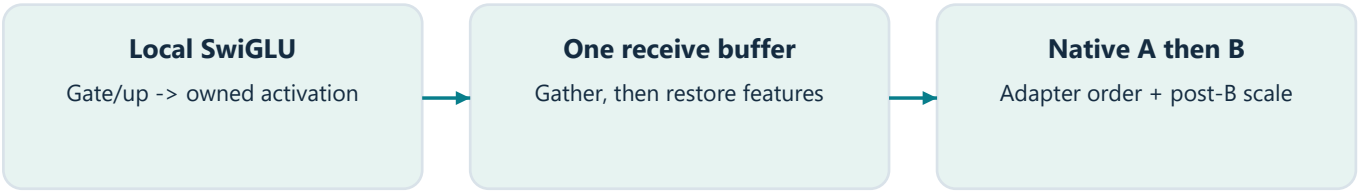
> **A supported kernel option is not the runtime selection**
> The module still contains register-scale, staged-K-scale, castpad centering, full-QK cache and alternate warp/thread options. Current callers do not enable them merely because the module was imported. In particular, TE uses the dedicated masked kernel, not the old unmasked TE screen winner.

Kitchen remains the actual DiT/Video default. The historical Video Group32 cast/pad winner was still about 1.60x the matched Kitchen API latency. Its 9.45% improvement over another Group32 implementation does not justify replacing Kitchen or claiming an integrated Video gain.

Source: S01, S08-S11, S49, S50, S53

# Save LoRA traffic without reordering arithmetic

The Native-ordered default also contains storage and lifetime optimizations.




**Owned delta view -> scale/add -> delete view + delta**

Do not retain one adapter's full B output into the next adapter.

For row LoRA, local SwiGLU runs before communication. Gathering the activated local features sends half as many values as gathering packed gate/up, while keeping the full Native A GEMM input. One all_gather_into_tensor receive buffer replaces a tensor-list gather plus concatenation. Rank-major receive storage is reordered into contiguous Native feature order before A; this conversion may allocate.

Where full B remains required, component-aware strided views select owned QKV or gate/up rows. Scaling is applied after B and addition follows adapter source order. Unused rows need no scale/add writes. The owned view and full delta are deleted before the next adapter's B GEMM so a view cannot prolong an entire preceding output allocation.

| Accuracy repair | Why it remains a contract |
|---|---|
| Do not pre-scale B | Moving strength before GEMM changes floating-point evaluation. |
| Do not concatenate A | Combining adapter ranks can change GEMM rounding; Concat A is retired and rejected. |
| Default gathers before A | Partial-A FP32 SUM changes grouping; only explicit Row Partial permits it. |
| Selective B is a separate guard | Only the accepted FC1 geometry replaces full B; other targets retain Native B dimensions. |

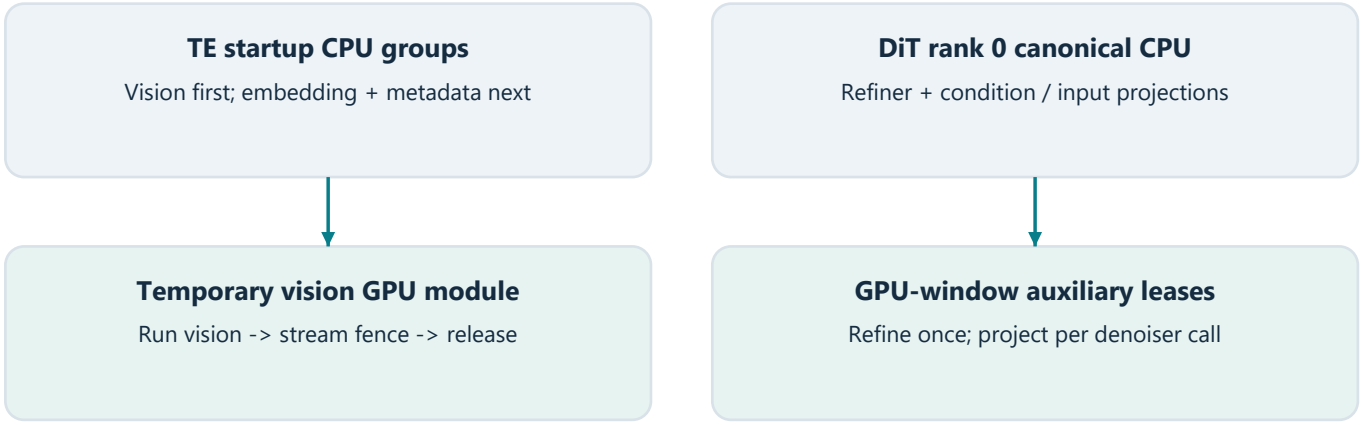
**Historical memory failure matters**

A rejected candidate retained the preceding full B through an owned view and reached 2244 MiB in the recorded multi-adapter case. The lifetime fix removed that extra overlap. Old streamed-mode timing/memory numbers do not describe the current removed streamed mode.

Source: S18, S42-S44, S33

# Stream the front end and auxiliary modules too

A transformer-only window misses large vision, embedding and refiner allocations.




TE startup has a separate bounded CPU cache for two groups: vision first, then embedding/metadata. Each can be admitted without first constructing all 50 decoder blocks. Raw checkpoint tensors transfer their host reservations into Native model construction; extra embedding-construction storage is admitted explicitly.

StagedVision keeps the canonical vision module on CPU and copies it only for the actual vision call. Its finally block synchronizes the current stream and releases the temporary GPU module, including Native cached FP32 projections, before the text decoder window starts. This is an intentional phase fence, not fully overlapped vision/TE execution.

DiT rank 0 owns condition projection, text refinement and video/audio patch projection. Rank 1 uses weightless placeholders and receives exact broadcast outputs. These auxiliary modules remain canonical on CPU; WeightSpecs lease them through the same GPU window. resident_backing=True prevents their existing CPU tensors being charged as another full construction.

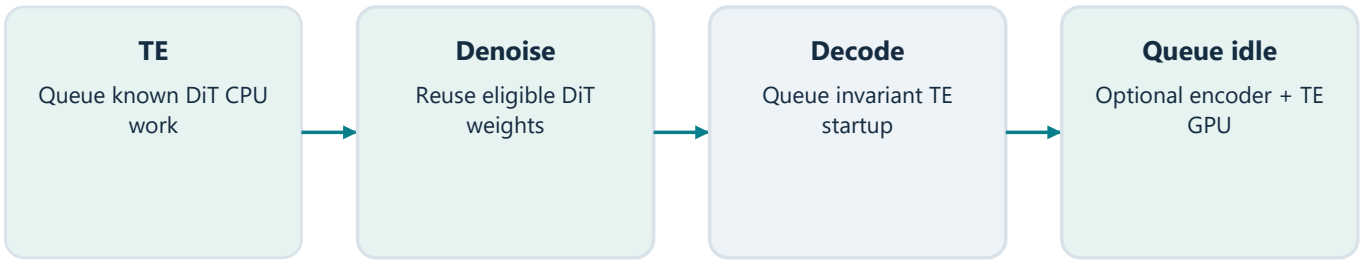
| When | Admitted auxiliary work |
|---|---|
| Before denoising | Native text refinement once, unless context is already hidden width 5376. |
| Each denoiser call | Native input projections, using the selected packing adapter; broadcast packed/projection output. |
| On release | Restore temporary bindings, break bound-method cycles, then release backing reservations. |

**Preserve the API branch**
Both _embed_and_pack and inline-projection adapters remain for existing qualified callers. The pinned local API determines the active branch. This document does not migrate either source or wrappers.

Source: S03-S05, S20, S22, S45

# Use known next work to hide preparation

CPU prefetch, GPU residency and queue-idle speculation have different start boundaries.




**Preparation, H2D and execution are separately admitted**

Next-prompt contents are never guessed; prompt start cancels idle speculation.

| Boundary | Current behavior and default |
|---|---|
| TE completes | cross_model_cpu=True can queue upcoming DiT CPU blocks; no speculative DiT CUDA allocation. |
| DiT evaluations | Retain eligible weights across actual model calls under capacity and next-use guards; this is separate from the two-group H2D horizon. |
| Public decode starts | cross_request_te=True queues model-invariant TE startup/vision CPU groups. With prefetch_lifecycle=True, also request decode_te_blocks (default 2) TE CPU blocks. |
| Decode completes | Without lifecycle mode, queue TE block preparation once startup work is no longer pending. |
| Queue becomes idle | Optional parent encoder preload, then bounded worker TE GPU preload. Cancel at the integrated prompt-start boundary. |

MiniMaxH3TP2Loader maps idle_residency to prefetch_lifecycle. Thus both are False in its ordinary defaults; programmatic callers can enable lifecycle scheduling independently. cross_request_te and cross_model_cpu remain True in RuntimeConfig. These controls must not be described as one always-enabled preloader.

**P0/P1 labels are overloaded**

The older loading plan calls direct rank loading P0 and independent CPU preparation P1. Later residency ablations call reuse-aware lifecycle P0 and pressure-aware admission P1. Use the explicit flags and source snapshot instead of transferring a P0/P1 result between experiments.

Source: S02, S03, S20, S22, S23, S45, S47, S48

# Preserve useful CPU weights under pressure

Deadline scheduling and cgroup policy supplement the byte budgets; they are not unbounded caches.

| Mechanism | Current contract |
|---|---|
| Preparation scheduling | One CPU preparation worker per cache, foreground priority at module boundaries, predicted next-use deadlines, independent pin/H2D stages. |
| Lifecycle option | reuse_aware follows prefetch_lifecycle. Upcoming entries and reload cost guide eviction; speculative far-future work cannot evict a nearer wanted entry. |
| Cgroup policy, optional | memory_pressure_aware=False by default. Linux cgroup-v2 observations account for nonfile usage, file-cache floor, pending preparation and PSI pressure. |
| Pressure admission | With the option on, speculative file-cache floor is 6 GiB; foreground protects up to 2 GiB. Shared reserve remains 3 GiB. There are at most two preparation leases across ranks. |
| No available cgroup metrics | Retain managed byte limits and system available-RAM checks. Do not fabricate Linux pressure data on Windows. |

CPU cache borrowing distinguishes an immutable prepared module from pin staging. Existing canonical auxiliary backing has already been charged. Speculative cancellation returns reservations only after real readers/copies no longer own the corresponding memory. Counters expose outcomes without retaining tensor payloads.

**Why this is not a solved I/O bottleneck**
The historical lifecycle/pressure study still approached 30 GiB cgroup usage and millions of refaults. Its TE gains did not reproduce as a large stable reduction. Later 22 storage controls verified 2,008 byte-correct block loads but promoted no new I/O algorithm. A file-cache floor cannot protect specific checkpoint pages.

Source: S03, S20, S22, S23, S46, S47, S55

# Measure a change against its own baseline

These retained results explain integration decisions; they are not additive pipeline gains.

| Historical physical experiment | Observed result | Boundary / limitation |
|---|---|---|
| Packed row preparation, same-round repeat | O: 33.760 -> 30.209 ms; FC2: 54.943 -> 47.612 ms | Complete API CUDA medians; synthetic current TP2 MNK; before newer SwiGLU/owned-FC2 defaults. |
| Packed FC2 Systems trace | 17 -> 10 launches/call | Removed buffer/max/quantizer work; INT32 SUM still about 19.162 ms in that separate capture. |
| Four NCCL channels | O: 35.154 -> 32.760 ms; FC2: 53.953 -> 51.630 ms | One allocation; fixed communicator setting; automatic protocol and no P2P retained. |
| Native-order LoRA storage | S8192 default FFN about 4.4-4.5% lower median | Previous correct implementation comparator; historical stack, adapter fixture and source. |
| Standalone idle pilot, 15 s gap | Reference+TE: 98.685 -> 81.108 s | Not a ComfyUI estimate: the pilot continued speculation into reference encoding after next request start. |

The row-fusion diagnostic retains rotation, GEMM, SUM and Native dequantization. Kernel averages come from separate observed calls and cannot be added to the warmed medians. The FC2 charts earlier in this guide are newer scoped memory studies, not another factor to multiply by these percentages.

**Version and quality gate**

Every row above predates at least part of the present combined source/stack. Matched operator byte gates support that experiment's implementation comparison. They do not establish current full-depth quality, near-capacity admission, queued concurrency, p95 latency or Linux qualification of the frozen local dependencies.

Source: S29, S30-S32, S40, S41, S43, S48, S50

# Do not promote experiments by name alone

The companion coverage matrix ties integrated features to live dispatch and records exclusions.

| Retained experiment family | Current disposition |
|---|---|
| INT8 PV / alternate thread mappings | Attention investigation only; current integrated TileLang PV remains FP16. |
| Packed-INT8 SUM / FP32 partial SUM | Not the default row transport. Scaled FP16 is now the default; Native INT32 remains the exact-reduction option. |
| Register/staged scales, castpad center, finite epilogue | Available experimental kernels, not automatically selected by current callers; Video stays Kitchen. |
| Concat A / streamed SwiGLU | Retired and rejected. Current optional Row Partial is distinct. |
| Cropped slabs / video TP/SP / temporal audio | Historical experiments or compatibility tools; current loader chooses aligned video queries and Native single-GPU FP32 stereo audio. |
| Preadv / mmap / descriptor advice / more workers | Storage-study candidates, not the production positional-read implementation. |
| ZSTD / Blosc2 / ZipNN / ZigZag-FSE | Separate CPU codec study, not active checkpoint decompression. |
| Latest API / unpinned Linux setup | Not an optimization to apply; preserve the validated checkout and exact package contract. |

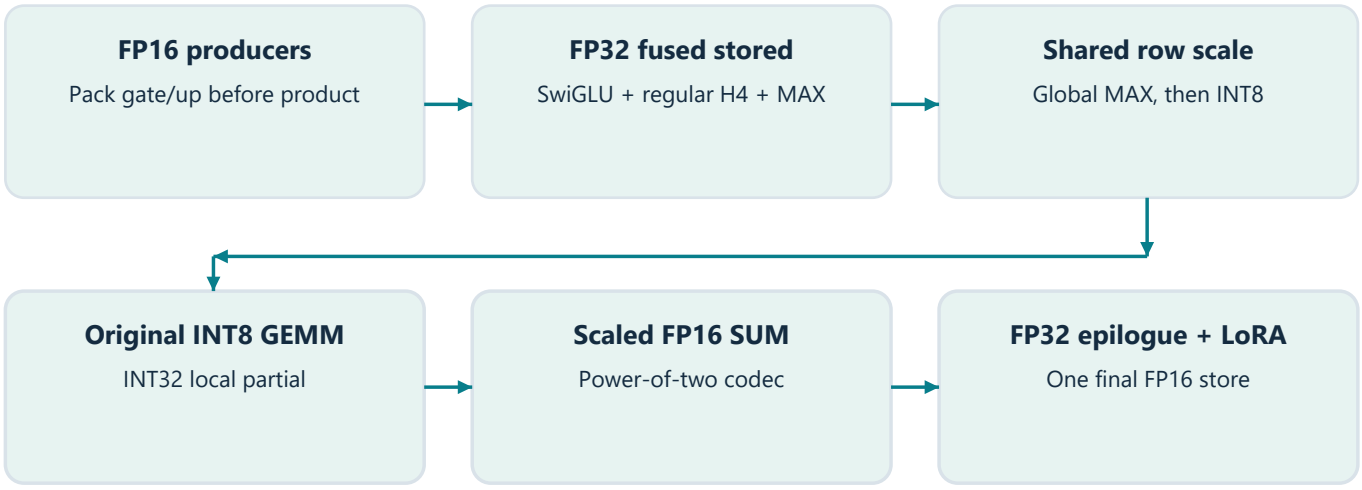
The coverage audit searches published documentation, benchmark/build sources and retained repository/external experiment records, then checks the current source call chain. Oversized raw traces, binaries and third-party dependency trees are outside this static text audit. A benchmark winner is treated as integrated only when the loader/runtime dispatch selects it.

**Audit deliverables**
docs/reports/H3_DOCUMENTATION_COVERAGE_AUDIT.md explains the omissions and decisions. docs/data/h3_documentation_coverage_20261020.json maps features, implementation symbols, source hashes, document coverage and scanned experiment families. No new model, GPU or profiler experiment was run.

Source: S01-S03, S39, S44, S49, S51, S52, S55, S56

# FP16 storage, FP32 nonlinear preparation

Promoted on 22 October: preserve the branch product before quantizing it.




The loader defaults to tensor_flow=fp16, residual divisor 64 for TE/DiT and zero extra gauges for vision/Video. Norm statistics, time/modulation paths, sampler state and final heads retain their explicit FP32 islands. Folding acts on private prepared weights/scales; INT8 checkpoint bytes are unchanged.

TE concatenates FP16 gate/up producers before activation. DiT already has packed producers. fused_stored loads them, computes FP32 SwiGLU and regular H4 ConvRot256, stores rotated FP32 values and group maxima, then obtains the shared row MAX across both ranks. INT8 quantization uses that scale before the original INT8-to-INT32 GEMM.

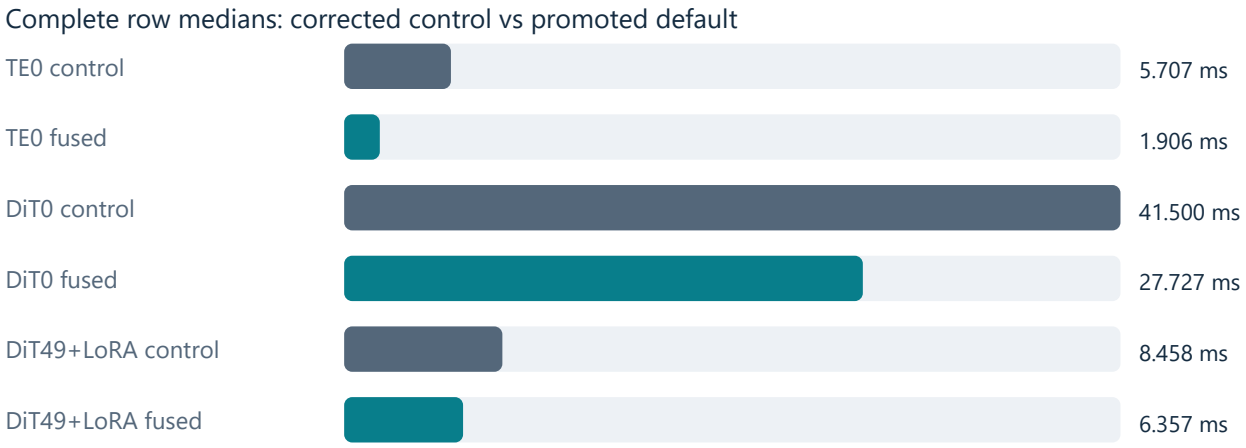
The existing power-of-two codec converts partial accumulators for FP16 AllReduce SUM. The TileLang epilogue reconstructs INT32 and dequantizes in FP32. Online LoRA is added in FP32 before a single FP16 output store; early casting could overflow before a cancelling adapter contribution.

**Controls remain explicit**
tilelang_packed selects INT32 SUM. owned_fp32/native restore prior preparation; fused_recompute is the measured memory alternative. Saved explicit widget values remain respected. The Native legacy flow is selectable.

Source: S01-S03, S06-S07, S14-S18, S57-S61

# Row wins are smaller block wins

18 configurations per rank; two real SM75 T4s, without an H3 sampler.




| Component | Row median control -> stored+scaled | Trained block median |
|---|---|---|
| TE0 S353 | 5.707 -> 1.906 ms (3.00x) | 24.584 -> 22.343 ms |
| DiT0 S6814 | 41.500 -> 27.727 ms (1.50x) | 265.846 -> 253.446 ms |
| DiT49 + LoRA S513 | 8.458 -> 6.357 ms (1.33x) | 39.012 -> 37.655 ms |

Control is corrected FP32 preparation with INT32 SUM. Measurements use two warmups, seven synchronized samples, slower-rank aggregation and separate compilation/loading. Fixed candidate order and variable clocks limit causal ranking of small differences. Recompute+scaled has the lowest S513 row median, 6.227 ms, while stored remains the requested default.

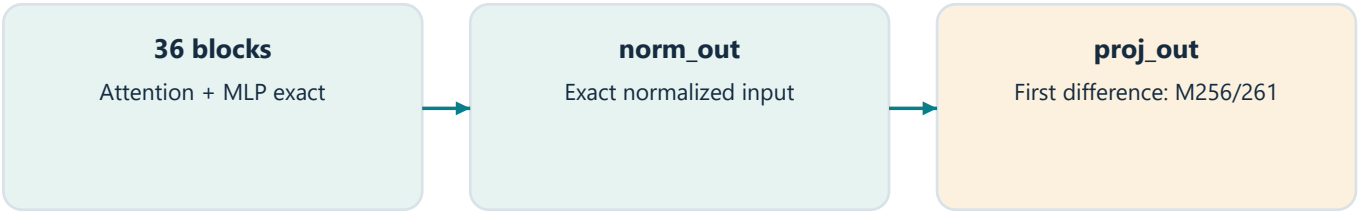
Fused codes disagree on 0.0003-0.0007% of entries; row relative L2 is about 6e-5 with INT32 and 4e-4 with scaled transport. No nonfinite outputs occurred. Stress-row traces retain LoRA AllGather at about 2.4-3.0 ms, limiting further acceleration.

**Evidence limits**
TE inputs are synthetic; DiT S6814 selects rows from retained S7895; layer49 reuses first-block input. Torch 2.11+cu128, Kitchen 0.2.37, TileLang 0.1.13. These are operator/component checks, not full generation or Windows-stack qualification.

Source: S57, S59-S61

# The 261-token mismatch starts at proj_out

Attention and MLP drift were ruled out in this trained decoder replay.




**Identical-input replay reproduces the mismatch**

The location is proven; the exact GEMM algorithm cause is not measured.

The 36-block Video decoder was checked at 74 boundaries per rank for split 256/5. All attention outputs, MLP outputs and final normalization match Native exactly. Rank 0 first differs at the final dense projection: M256 versus Native M261, K2048 to N3072. Identical normalized-input replay reproduces the difference.

| Boundary | Observed result |
|---|---|
| Raw decoder tensor | 145,205 / 786,432 values differ; max absolute difference 0.00390625. |
| Valid last image frame | 33,668 / 196,608 values differ; max absolute difference 0.0004472733. |
| Rank 1 suffix | All 74 boundaries match; five suffix tokens are discarded from valid pixels. |
| Narrow repair to test | Gather final hidden rows and evaluate Native full-M projection on rank 0. Not implemented or measured in this batch. |

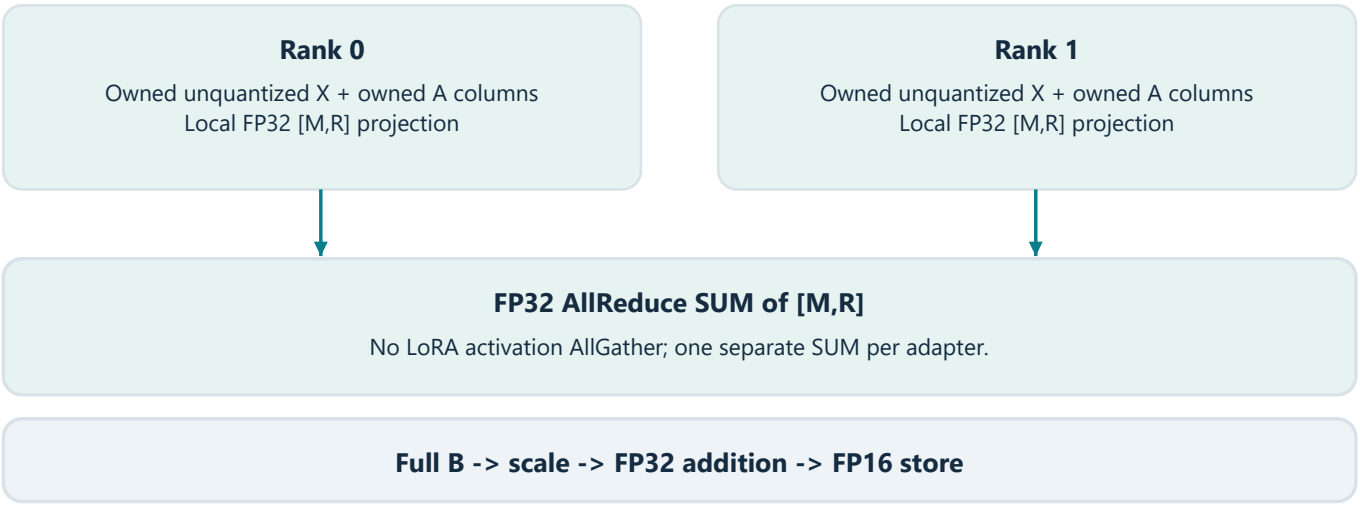
**Default promotion does not repair Video identity**

The FP16-fix/fusion/transport promotion is separate from Video query partitioning. Shape-dependent final-projection arithmetic is localized; the exact backend reduction/algorithm cause remains unmeasured. No full-pipeline visual or audio acceptance is asserted.

Source: S25, S59-S61

# SUM the low-rank A intermediate

The default avoids gathering full LoRA activations; hidden SUM stays FP32.




```
H = SUM_p linear(X_owned[p], A_owned[p])  # FP32 SUM
Y = Y_base_fp32 + linear(H, B_full) * strength
Y_storage = FP16(Y)                       # one store
```

The loader now defaults lora_row_partial_reduce=True. Only row-parallel main DiT adapters read owned A columns. FC2 uses unquantized, unrotated FP32 SwiGLU values. Each adapter retains its own A GEMM, SUM, full B GEMM and post-B scale/add order. Column-parallel Selective B and unsharded refiner adapters are unchanged.

| DiT projection, rank128 adapter | Local K / hidden R | Input tensor byte ratio |
|---|---|---|
| Attention O | 3584 / 128 | 28x |
| FC2 | 7168 / 128 | 56x |

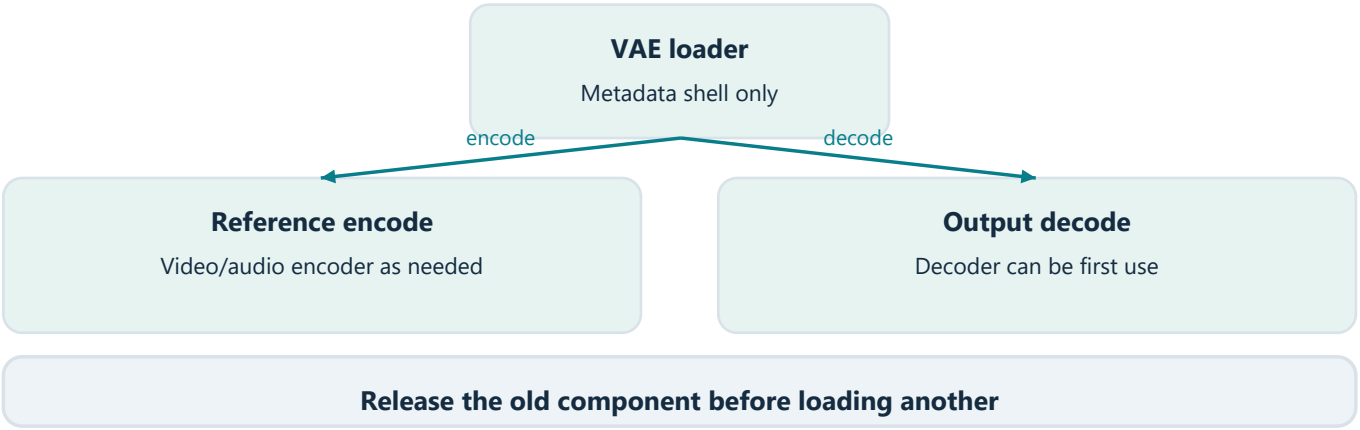
At M6814, one FC2 owned FP32 activation is 186.32 MiB versus 3.33 MiB for a rank128 hidden. Two such adapters halve the aggregate ratio to 28x. These are tensor-size calculations, not observed NCCL speedups. Full B and other base collectives remain.

**Precision and evidence**
Hidden reduction is FP32, independent of scaled-FP16 base transport. Partial summation changes Native grouping; set False to restore full-input gathering. Four combined local logical-owner GPU checks match an independent partial/codec reference. New T4 latency and generation quality remain unmeasured.

Source: S03, S07, S18, S63-S65

# Load only the encoder a reference uses

Metadata-only VAE shells avoid unconditional video and audio weight reads.




| Actual reference use | Video encoder | Audio encoder |
|---|---|---|
| Text only / output decode only | No | No |
| Image or silent video | On encode | No |
| Standalone audio | No | On encode |
| Video with matched soundtrack | On encode | On encode |
| Image/video + standalone audio | On encode | On encode |

Corresponding VAE connections are required; Native TE-only references remain supported. The first encode/decode call loads its needed component. Decoder-first use bypasses encoder reads. Existing locks, host-budget leases and stage teardown prevent component overlap; sampling releases reference encoders.

Idle prefetch follows known H3 reference/encode consumers, including nested or flattened reference lists. It does not infer reference requirements merely from the presence of output VAE loaders. Unknown/dynamic producers retain demand-loading correctness; graph analysis only supplies prefetch hints.

**Checks and boundaries**
64 focused tests pass. Trained local Video and Audio VAE encode/decode checks, including decoder-first use, match Native with zero constructor component payload. Combined row checks use local SM120 logical transport; no new Kaggle job or full H3 pipeline was run.

Source: S03, S24, S62-S66

# Implementation and evidence catalogue

Paths are relative to the root on the cover. Embedded hashes identify the actual source bytes.

**S01** patches/h3_operator_policy.py
Operator defaults, architecture guards and refiner SDPA

**S02** custom_nodes/minimax_h3_tp2/__init__.py
Node controls and service identity

**S03** patches/comfy_tp_runtime.py
Persistent workers, runtime budgets and phase protocol

**S04** patches/comfy_tp_models.py
Native model construction and staged startup

**S05** patches/comfy_sampler.py
Native conditioning/sampling and public decode adapters

**S06** patches/tp_qwen3vl.py
TE TP2 geometry, attention boundary and tensor lifetimes

**S07** patches/tp_dit.py
DiT TP2 geometry, AdaLN and residual order

**S08** patches/tilelang_group32_masked.py
Mask-aware Group32 TE kernel

**S09** patches/tilelang_grouped_tp_kitchen_v2.py
Dense preparation, query caching and kernel mapping

**S10** patches/tilelang_grouped_tp_optimized.py
Sparse preparation, tau0 routing and reference equations

**S11** patches/tilelang_attention_p2.py
Omitted-only centroid score workspace

**S12** patches/tilelang_row_quant.py
Global-MAX activation quantizer and packed row preparation

**S13** patches/tilelang_swiglu_pad.py
SwiGLU fusion, lifetime repair and owned FP32 rotation

**S14** patches/tilelang_transport_p2.py
Default scaled FP16 transport and fused dequantization

**S15** patches/tilelang_transport_tradeoffs.py
Immutable transport codec and power-of-two bound

**S16** patches/parallel_int8.py
INT8 GEMM, INT32 SUM and native dequantization reference

**S17** patches/optimized_tp.py
Native/SM75 rotation dispatch

**S18** patches/h3_lora.py
Ordered online adapters, selective B and row partials

Source: Current source audit: 2026-10-22

# Implementation and evidence catalogue

Paths are relative to the root on the cover. Embedded hashes identify the actual source bytes.

**S19** patches/offline_lora.py
Whole-layer dual-GPU baking and atomic publication

**S20** patches/windowed_tp.py
Positional reads, streamed specs and model proxies

**S21** patches/rank_local_int8.py
Owned checkpoint ranges and optional rank-local exports

**S22** patches/host_weight_cache.py
Shared host budget, pageable cache and pin staging

**S23** patches/weight_residency.py
H2D events, leases, eviction and adaptive lookahead

**S24** patches/staged_vae.py
Encoder/decoder-only construction and nonpersistent buffers

**S25** patches/parallel_vae.py
Video query ownership, ABI gates and decoder replicas

**S26** patches/audio_channel_decode.py
Historical retired audio-channel experiment

**S27** patches/comfy_idle_lifecycle.py
Queue-idle admission, pressure and cancellation hooks

**S28** patches/legacy_runtime.py
Pinned source selection and source identity checks

**S29** runtime/local_conda.json
Frozen user environment and ComfyUI identity

**S30** docs/reports/FC2_QUANTIZER_MEMORY.md
FC2 memory study, qualification scope and activation defaults

**S31** docs/data/fc2_quantizer_t4_v1_summary.json
Physical T4 FC2/FFN measurements at S513/8192/16384

**S32** docs/data/fc2_quantizer_t4_v2_summary.json
Integrated physical T4 S32768 confirmation

**S33** docs/guides/H3_OPERATOR_INTEGRATION.md
Historical target gates and integration boundaries

**S34** docs/guides/H3_OPERATOR_ENHANCEMENTS.md
Transport/lifetime/read/refiner enhancement qualification

**S35** docs/guides/H3_LOADING_STAGES.md
LoRA bake, VAE component read sizes and idle encoder stages

**S36** docs/guides/PARALLEL_VAE_INTEGRATION.md
Public decode preservation and audio numerical limits

Source: Current source audit: 2026-10-22

# Implementation and evidence catalogue

Paths are relative to the root on the cover. Embedded hashes identify the actual source bytes.

**S37** docs/data/H3_PIPELINE_REVIEW_RESULTS.json
Recorded pipeline review checks and source closure

**S38** docs/reports/TP2_P2_OPTIMIZATION_REPORT.md
Historical profiling, scalar workspace and transport caveats

**S39** docs/reports/H3_RUNTIME_PROVENANCE_AUDIT.md
Local source provenance and Linux qualification limits

**S40** docs/reports/REF2VA_T4_RESULTS.md
Historical Native GEMM, fixed transport and scalemax attribution

**S41** docs/reports/TP2_ROW_FUSION_RESULTS.md
External-max, strided-read and packed-row staged ablations

**S42** docs/reports/ONLINE_LORA_NATIVE_ORDER_REPAIR.md
Controlled adapter rounding causes and Native-order repair

**S43** docs/reports/ONLINE_LORA_BYTE_EXACT_OPTIMIZATION.md
Single-buffer gather and owned-delta lifetime qualification

**S44** docs/reports/ROW_PARTIAL_MEMORY.md
Current Row Partial staging, bounds and retired choices

**S45** docs/reports/LOADING_OPTIMIZATION.md
Rank ranges, CPU scheduling and decode-start startup prefetch

**S46** patches/host_memory_pressure.py
Read-only cgroup-v2 pressure inputs and fallback availability

**S47** docs/reports/TE_RESIDENCY_P01_RESULTS.md
Historical lifecycle/pressure ablations and limited TE gains

**S48** docs/reports/TE_IDLE_RESIDENCY_RESULTS.md
Idle benchmark versus integrated prompt-start boundary

**S49** docs/reports/TP2_KITCHEN_GROUP32_V2_RESULTS.md
Unpromoted cast/pad, scale staging and finite-output candidates

**S50** docs/reports/TP2_OPERATOR_PASS2.md
Attention preparation, direct stores and NCCL channel evidence

**S51** docs/reports/H3_DOCUMENTATION_COVERAGE_AUDIT.md
Current integration coverage and historical experiment reconciliation

**S52** docs/data/h3_documentation_coverage_20261020.json
Machine-readable feature/source and experiment-family coverage

**S53** patches/tilelang_grouped_fused.py
Sparse fused quantizer and rounded preparation boundaries

**S54** docs/reports/TP2_OPERATOR_RESULTS.md
Early TP2 operator screen and limited Video microchain scope

Source: Current source audit: 2026-10-22

# Implementation and evidence catalogue

Paths are relative to the root on the cover. Embedded hashes identify the actual source bytes.

**S55** docs/reports/TE_IO_ABLATION_RESULTS.md
Storage controls: validated bytes without a promoted I/O winner

**S56** docs/reports/TE_COMPRESSION_LOCAL_20261020.md
CPU codec experiment, not an integrated loader stage

**S57** patches/tilelang_swiglu_quant.py
Promoted FP32 regular-H4 SwiGLU preparation

**S58** patches/h3_tensor_flow.py
FP16 residual fix, private folding and FP32 stability islands

**S59** docs/data/fp16_fc2_fusion_video_split_20261022.json
Physical dual-T4 matrix, traces and Video projection localization

**S60** docs/reports/FP16_FC2_FUSION_VIDEO_SPLIT_20261022.md
Measured limits and authorized promotion

**S61** docs/reports/H3_FP16_FUSED_DEFAULT_20261022.md
Production dispatch, rollback and integration checks

**S62** patches/h3_reference_plan.py
Reference-sensitive idle encoder hints

**S63** docs/reports/H3_ROW_PARTIAL_REFERENCE_LOADING_20261022.md
Row-A partial default and lazy reference loading

**S64** docs/data/h3_row_partial_reference_loading_20261022.json
Local integration and trained lazy-VAE evidence

**S65** benchmarks/test_h3_row_partial_combined.py
Independent FP32 hidden SUM and factor-shard checks

**S66** benchmarks/test_h3_reference_loading.py
Reference-type and decoder-first lifecycle checks

Source: Current source audit: 2026-10-22